HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP LỚN

MÔN: TÌM KIẾM VÀ TRUY XUẤT THÔNG TIN

Đề tài: No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval

Giảng viên hướng dẫn	:	**TS. Đào Thị Thúy Quỳnh**
Ngành	:	**Hệ thống thông tin**
Lớp	:	**M25CQHT02-B**
Học viên nhóm 9	:	**Nguyễn Văn Toàn - B25CHHT115**
	:	**Phạm Văn Thọ - B25CHHT114**
	:	**Hà Phương Nguyên - B25CHHT108**
	:	**Trần Duy Khánh - B25CHHT100**
	:	**Khamsing Outhaihueng - B25CHHT125**

Hà Nội - Tháng 10 năm 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	Học viên thực hiện
1	Điều phối, tổng hợp và biên tập báo cáo; Chương 1 “Bài toán, bối cảnh và mục tiêu”	Nguyễn Văn Toàn - B25CHHT115
2	Dịch bài báo; Chương 2 “Biểu diễn sparse và huấn luyện SSR” phần biểu diễn sparse, sparse MaxSim và cận sai lệch (mục 2.1–2.5); Phụ lục B	Phạm Văn Thọ - B25CHHT114
3	Dịch bài báo; Chương 2 “Biểu diễn sparse và huấn luyện SSR” phần hybrid training và đối chiếu thiết lập huấn luyện với mã nguồn (mục 2.6–2.8)	Hà Phương Nguyên - B25CHHT108
4	Cài đặt môi trường, chạy lại mã nguồn và checkpoint, rà soát và sửa lỗi, tối ưu kernel truy hồi, đo so với ColBERTv2/PLAID; Chương 3 “Lập chỉ mục và truy hồi”; Chương 4 “Tái lập và kiểm chứng” (mục 4.2–4.7); Chương 5 “Cải tiến, thảo luận và kết luận” (mục 5.1–5.5); Phụ lục A, D, E	Trần Duy Khánh - B25CHHT100
5	Chương 4 “Tái lập và kiểm chứng” phần kết quả công bố của bài báo (mục 4.1) và Phụ lục C; Chương 5 “Cải tiến, thảo luận và kết luận” phần thảo luận và kết luận (mục 5.6–5.7); làm slide trình bày	Khamsing Outhaihueng - B25CHHT125

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	Học viên	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Nguyễn Văn Toàn - B25CHHT115	5	5	5	5	5
2	Phạm Văn Thọ - B25CHHT114	5	5	5	5	4
3	Hà Phương Nguyên - B25CHHT108	5	5	5	5	4
4	Trần Duy Khánh - B25CHHT100	5	5	5	5	4
5	Khamsing Outhaihueng - B25CHHT125	5	5	5	5	4

Ghi chú: thang điểm 0–5 cho từng tiêu chí (0: không tham gia / không hoàn thành; 5: chủ động, tích cực / hoàn thành xuất sắc).

TÓM TẮT

Multi-Vector Retrieval (MVR) theo kiểu ColBERT giữ một vector cho mỗi token và so khớp bằng MaxSim, nhờ đó chính xác hơn các mô hình một vector, nhưng ColBERTv2/PLAID phải dựa vào K-means clustering trên hàng tỷ vector token, khiến lập chỉ mục tốn hàng trăm giờ và không thể thêm tài liệu mà không dựng lại. Báo cáo tìm hiểu bài báo *No More K-means* (ICML 2026) và phương pháp Single-stage Sparse Retrieval (SSR): mỗi embedding token được TopK Sparse Autoencoder chiếu lên không gian 16.384 chiều chỉ có 32 chiều khác không, mỗi neuron hoạt động được xem như một pseudo-token, và truy hồi diễn ra trong một giai đoạn trên inverted index.

Phần lý thuyết trình bày sparse MaxSim và cận sai lệch so với điểm dense, hàm mục tiêu hybrid training, inverted index mức neuron và thuật toán coarse-to-fine SSR++.

Phần thực nghiệm kiểm chứng mã nguồn và checkpoint công bố trên NFCorpus, SciFact, FiQA, Quora và MS MARCO, với ColBERTv2 làm đối chứng trên cùng pipeline. Nhóm phát hiện 10 điểm sai lệch giữa mã nguồn và bài báo, đã xử lý 9 điểm bằng bản sửa hoặc bổ sung cấu hình; cơ chế `token_skip_mode` ghi trong config của checkpoint chưa được triển khai. Riêng lỗi công thức chấm điểm trên inverted index làm nDCG@10 trên NFCorpus rơi từ 43,97 xuống 8,33. Checkpoint công bố vượt ColBERTv2 trên split test của ba tập BEIR đã đo, nhưng kém ColBERTv2 trên các split train/dev đối chứng của cùng tập và trên MS MARCO theo giao thức re-ranking BM25 top-100; các phép kiểm tra hiện có chưa xác định được nguyên nhân của chênh lệch này.

Phần cải tiến gồm kernel truy hồi exact `fast_hyb` cho kết quả xếp hạng giống hệt bản gốc và nhanh hơn 29,6 lần so với bản exact của repo trên cùng 100 query FiQA (1.635 ms so với 55 ms mỗi query, CPU 12 luồng), một bản exact trên GPU, và phép xấp xỉ LB-rerank giữ 99,7% top-10 của phép tính exact với 0,67 lần thời gian trên FiQA dev. Đổi lại, chỉ mục của bản tối ưu lớn hơn chỉ mục của repo khoảng 1,6 lần.

Từ khóa: Multi-Vector Retrieval, Sparse Autoencoder, inverted index, late interaction, ColBERT, coarse-to-fine pruning, tái lập nghiên cứu.

MỤC LỤC

TÓM TẮT . . . . . **ii**

DANH MỤC CÁC TỪ VIẾT TẮT . . . . . **vii**

1 Bài toán, bối cảnh và mục tiêu **1**
1.1 Bài toán truy hồi thông tin 1
1.2 Ba hướng tiếp cận truy hồi 1
1.3 Pipeline ba tầng của ColBERTv2/PLAID 2
1.4 Hạn chế của hướng tiếp cận dựa trên K-means 3
1.5 Ý tưởng của SSR và bài báo 3
1.6 Các nghiên cứu liên quan 4
1.7 Mục tiêu, câu hỏi kiểm chứng, phạm vi và phương pháp 4

2 Biểu diễn sparse và huấn luyện SSR **6**
2.1 Từ biểu diễn dense sang biểu diễn sparse 6
2.2 TopK Sparse Autoencoder 6
2.3 Late interaction trên không gian sparse 7
2.4 Hai biến thể SSR-tok và SSR-CLS 8
2.5 Cận sai lệch so với điểm dense 8
2.6 Hàm mục tiêu hybrid training 9
2.7 Vai trò của từng thành phần theo ablation của bài báo 11
2.8 Đối chiếu thiết lập huấn luyện với mã nguồn công bố 11

3 Lập chỉ mục và truy hồi **13**
3.1 Inverted index mức neuron 13
3.2 Truy hồi một giai đoạn (SSR) 14
3.3 Truy hồi coarse-to-fine (SSR++) 14
3.4 Độ phức tạp 15
3.5 Những điểm cần chú ý khi cài đặt 16

4 Tái lập và kiểm chứng **17**
4.1 Những kết quả công bố cần kiểm chứng 17
4.2 Môi trường, checkpoint, dữ liệu và giao thức đo 19
4.3 Kiểm tra pipeline bằng baseline và các phép tính độc lập 20
4.4 Sai lệch mã nguồn và trạng thái xử lý 20
4.5 Kết quả trên split test và đối chứng train/dev 21
4.6 MS MARCO theo giao thức re-ranking BM25 top-100 23
4.7 So sánh với ColBERTv2/PLAID và giới hạn của phép so sánh 23

5 Cải tiến, thảo luận và kết luận **25**
5.1 Nút thắt đã đo được và mục tiêu tối ưu 25
5.2 Kernel exact `fast_hyb` và các thay đổi cấu trúc dữ liệu 26

5.3 Kết quả trên cùng tập query: một luồng, nhiều luồng, GPU . . . 26
5.4 Phương án xấp xỉ LB-rerank . . . 28
5.5 Những hướng đã thử nhưng không hiệu quả . . . 28
5.6 Thảo luận . . . 28
5.7 Kết luận và hướng phát triển . . . 30

TÀI LIỆU THAM KHẢO . . . 31

A Chi tiết các sai lệch trong mã nguồn và bản sửa . . . 33
A.1 Điểm 2: nạp checkpoint tạo SAE ngẫu nhiên . . . 33
A.2 Điểm 1: công thức chấm điểm trên inverted index . . . 33
A.3 Điểm 3 và 4: ánh xạ neuron và bước thô SSR++ . . . 34
A.4 Điểm 5–7 và 9: độ đo và huấn luyện . . . 34
A.5 Điểm 8: dựng chỉ mục với corpus lớn . . . 35
A.6 Điểm 10: `token_skip_mode` . . . 35
A.7 Trích log của các lần chạy được dẫn trong thân bài . . . 35

B Chứng minh Định lý A và Định lý B . . . 37

C Các kết quả khác của bài báo . . . 38
C.1 Bộ dữ liệu . . . 38
C.2 Bảng 1 đầy đủ . . . 38
C.3 Backbone LLM, LoTTE, văn bản dài và LIMIT . . . 38
C.4 Hiệu quả trên CPU và theo quy mô dữ liệu . . . 38
C.5 Siêu tham số h, K và trọng số loss . . . 41

D Nguồn của các số liệu hiệu năng . . . 43

E Hướng dẫn chạy lại thí nghiệm . . . 44
E.1 Cấu trúc mã nguồn công bố . . . 44
E.2 Cài đặt và chạy . . . 44

DANH MỤC CÁC HÌNH ẢNH

1.1 Hai cách biểu diễn tài liệu: SVR và MVR 2
1.2 Pipeline của ColBERTv2/PLAID và của SSR 3

2.1 Luồng biến đổi một token trong SSR 6
2.2 Tổng quan quá trình huấn luyện SAE trong SSR 9

3.1 Cấu trúc inverted index mức neuron 14
3.2 Sơ đồ SSR++ 14

4.1 Chất lượng BEIR so với độ trễ theo bài báo 18
4.2 Chất lượng trên split đối chứng và split test (nhóm đo) 22

5.1 Pipeline truy hồi của SSR và vị trí các tối ưu 25
5.2 Độ trễ truy vấn trên cùng 100 query FiQA (nhóm đo) 27

C.1 Hình 3 của bài báo 40
C.2 Hình 4 của bài báo 41

DANH MỤC CÁC BẢNG

1.1 So sánh ba họ phương pháp truy hồi: BM25, SVR và MVR (ColBERT). 2
1.2 Hạn chế của phương pháp dựa trên K-means và cách SSR giải quyết, kèm vị trí trình bày trong báo cáo. 3

2.1 So sánh biểu diễn dense của ColBERT và biểu diễn sparse của SSR. 6
2.2 Siêu tham số huấn luyện SAE với backbone BERT-base theo Bảng 6 của bài báo. 7
2.3 Ví dụ tính tay điểm sparse cho token truy vấn q_1 với hai token tài liệu. 7
2.4 Các giả thiết của phân tích sai lệch bị chặn. 8
2.5 Các thành phần của hàm loss và vai trò của từng thành phần. 10
2.6 Ablation từng thành phần của hàm loss theo bài báo (nDCG@10 trung bình BEIR, $K = 32$). . . . 11
2.7 Thiết lập huấn luyện: bài báo, mặc định của mã nguồn công bố, và trả lời của tác giả trong issue. Cột cuối là cách xử lý của nhóm. 11

3.1 So sánh độ phức tạp giữa PLAID, SSR và SSR++. 16

4.1 Các câu hỏi kiểm chứng, phép kiểm tra, kết quả chính và giới hạn. 17
4.2 Bảng 1 của bài báo (trích) 18
4.3 Chi phí lập chỉ mục trên MS MARCO theo bài báo. 18
4.4 So sánh SSR và SSR++ trên MS MARCO passage theo bài báo (Bảng 5). 19

4.5 Cột MS MARCO của Bảng 1 theo từng phiên bản của bài báo. 19
4.6 Môi trường thực nghiệm của nhóm. 19
4.7 Các tập dữ liệu nhóm đã dùng. 20
4.8 Tái lập số đã công bố của các baseline bằng pipeline của nhóm. 20
4.9 Các sai lệch giữa mã nguồn công bố và bài báo, kèm trạng thái xử lý của nhóm. 21
4.10 nDCG@10 trên split test (nhóm đo) 21
4.11 nDCG@10 trên split đối chứng và split test (nhóm đo) 22
4.12 Re-ranking BM25 top-100 trên MS MARCO (nhóm đo) 23
4.13 Hiệu năng trên FiQA (nhóm đo) 23
4.14 Đối chiếu từng tuyên bố về hiệu quả của bài báo với số nhóm đo trên FiQA. 24

5.1 Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu (nhóm đo trên FiQA). 26
5.2 Độ trễ của các kernel exact trên cùng tập query (nhóm đo) 27
5.3 Phân bố độ trễ trên toàn bộ 648 query FiQA test (nhóm đo, ms/query). 27
5.4 Các phương án xấp xỉ trên FiQA dev (nhóm đo) 28
5.5 Độ chặt của cận trên dùng để pruning (nhóm đo) 28

C.1 Các bộ dữ liệu dùng trong đánh giá của bài báo. 38
C.2 Bảng 1 của bài báo (đầy đủ) 39
C.3 Kết quả theo bài báo khi dùng Llama-embed-nemotron-8b làm backbone đóng băng (trung bình nDCG@10 trên 14 tập). 40
C.4 Success@5 trên LoTTE theo bài báo cho hai loại truy vấn Search và Forum. 40
C.5 Kết quả trên bộ LIMIT theo bài báo. 40
C.6 So sánh hiệu quả trên CPU theo bài báo (MS MARCO passage; chỉ mục tính bằng giờ, truy hồi tính bằng ms/truy vấn). 40
C.7 nDCG@10 theo mức sparsity K và theo nhóm miền dữ liệu (theo bài báo). 41
C.8 K cố định và K thích ứng theo độ dài truy vấn (theo bài báo). 41
C.9 Độ nhạy của nDCG@10 trung bình BEIR theo ba trọng số α, β, γ (theo bài báo). 42

D.1 Nguồn của các số liệu hiệu năng (nhóm đo). 43

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	**Cụm từ tiếng Anh**	**Giải thích**
ANN	Approximate Nearest Neighbor	Tìm láng giềng gần đúng
BEIR	Benchmarking IR	Bộ 18 tập dữ liệu đánh giá truy hồi zero-shot
BM25	Best Matching 25	Hàm xếp hạng từ vựng cổ điển
CE	Cross-Entropy	Mất mát entropy chéo
CLS	Classification token	Token đặc biệt đứng đầu chuỗi của BERT
IR	Information Retrieval	Truy xuất thông tin
LB	Lower Bound	Cận dưới (dùng trong phép xấp xỉ LB-rerank của nhóm)
MaxSim	Maximum Similarity	Phép tương tác muộn của ColBERT
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
MVR	Multi-Vector Retrieval	Truy hồi đa vector (mỗi tài liệu là nhiều vector token)
nDCG	normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
PLAID	Performance-optimized Late Interaction Driver	Bộ máy truy hồi của ColBERTv2 dùng K-means và nén residual
SAE	Sparse Autoencoder	Bộ tự mã hóa thưa
SSR	Single-stage Sparse Retrieval	Truy hồi thưa một giai đoạn (phương pháp của bài báo)
SSR++	SSR with coarse-to-fine pruning	SSR có thêm bước lọc thô rồi chấm tinh
SVR	Single-Vector Retrieval	Truy hồi đơn vector
TopK	Top-K activation	Chỉ giữ K giá trị lớn nhất, còn lại đặt bằng 0
UB	Upper Bound	Cận trên (của một khối posting)
VRAM	Video RAM	Bộ nhớ của GPU

CHƯƠNG 1. BÀI TOÁN, BỐI CẢNH VÀ MỤC TIÊU

Chương này đặt bài toán truy hồi thông tin, đối chiếu ba họ phương pháp (lexical retrieval, Single-Vector Retrieval và Multi-Vector Retrieval), chỉ ra vì sao bước K-means clustering trong các hệ MVR hiện đại trở thành nút thắt, rồi giới thiệu ý tưởng của SSR và bài báo. Phần cuối nêu các câu hỏi kiểm chứng mà báo cáo sẽ trả lời, cùng phạm vi và phương pháp thực hiện.

1.1. Bài toán truy hồi thông tin

Cho một truy vấn Q và một kho tài liệu $\mathcal{D} = \{D_1, \ldots, D_{|\mathcal{D}|}\}$, trong đó Q và mỗi D đều là chuỗi token. Hệ truy hồi cần trả về top-k tài liệu xếp hạng theo mức liên quan với Q, dựa trên một hàm chấm điểm $s_\theta(Q, D) \in \mathbb{R}$ được học sao cho tài liệu liên quan D^+ luôn có điểm cao hơn tài liệu không liên quan D^-:

$$s_\theta(Q, D^+) > s_\theta(Q, D^-). \tag{1.1}$$

Khác biệt giữa các họ phương pháp nằm ở cách biểu diễn Q và D, và ở cấu trúc dữ liệu cho phép tìm top-k mà không phải chấm điểm toàn bộ kho. Một hệ thống thực tế được đánh giá đồng thời theo ba tiêu chí: chất lượng xếp hạng, đo bằng MRR@10 hoặc nDCG@10 (định nghĩa ở mục 4.2.1); độ trễ cho mỗi truy vấn; và chi phí lập chỉ mục, gồm thời gian, bộ nhớ đỉnh, kích thước chỉ mục và khả năng thêm tài liệu mới mà không phải dựng lại. MRR@10 chỉ xét tài liệu đúng đầu tiên, còn nDCG@10 xét cả thứ tự và mức liên quan của mười kết quả đầu, nên hai độ đo này không so sánh trực tiếp được với nhau. Điểm này sẽ quan trọng khi đọc các bảng của bài báo (mục 4.1.4).

1.2. Ba hướng tiếp cận truy hồi

1.2.1. Lexical retrieval với BM25 và inverted index

Lexical retrieval so khớp từ một cách chính xác. Mỗi từ t trong kho được gắn với một *posting list* $I_t = \{(D, w_{D,t})\}$ liệt kê các tài liệu chứa nó cùng trọng số; cấu trúc dữ liệu này gọi là *inverted index*. BM25 [15] chấm điểm tài liệu bằng tổng có trọng số của tần suất từ, có chuẩn hóa theo độ dài tài liệu và độ hiếm của từ. Ưu điểm lớn nhất là tốc độ: truy vấn chỉ cần duyệt $|Q|$ posting list và cộng dồn điểm, chi phí tỉ lệ với tổng độ dài các posting chứ không với kích thước kho. Inverted index còn hỗ trợ safe pruning (WAND, Block-Max WAND): mỗi block posting lưu cận trên của trọng số để bỏ qua những block không thể đưa tài liệu vào top-k. Nhược điểm là không hiểu đồng nghĩa: hai câu diễn đạt cùng một ý bằng từ khác nhau sẽ không khớp. Inverted index cũng là hạ tầng mà SSR tái sử dụng ở Chương 3.

1.2.2. Single-Vector Retrieval (SVR)

SVR nén toàn bộ tài liệu thành một vector, thường lấy từ token `[CLS]` hoặc bằng pooling, rồi tính điểm bằng tích vô hướng với vector truy vấn; tìm kiếm dùng các thuật toán approximate nearest neighbor (ANN) như HNSW hay IVF. Hạn chế căn bản là mọi chi tiết mức token bị nén vào một vector kích thước cố định. Benchmark LIMIT [14] cho thấy rõ điều này: các mô hình SVR mạnh chỉ đạt Recall@5 dưới 5%, trong khi các mô hình MVR đạt trên 70% (Bảng C.5, Phụ lục C).

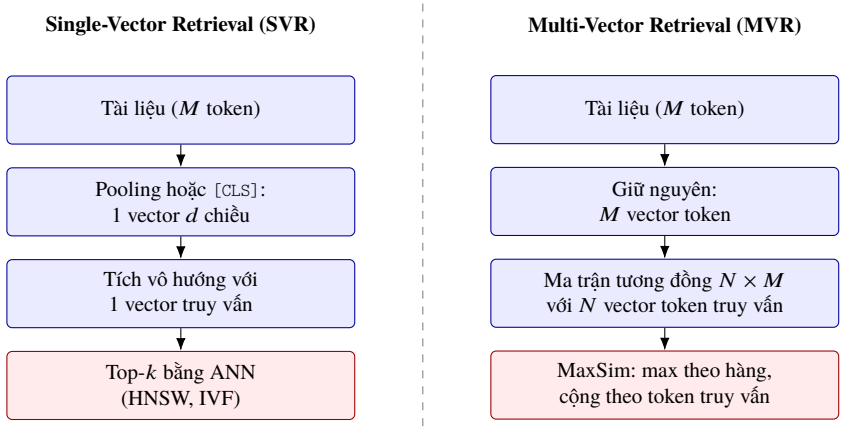
1.2.3. Multi-Vector Retrieval (MVR) và late interaction

MVR giữ nguyên một embedding cho mỗi token của cả truy vấn lẫn tài liệu. ColBERT [2] mã hóa query thành $\{q_1, \ldots, q_N\}$ và tài liệu thành $\{d_1, \ldots, d_M\}$ (mỗi token một vector 128 chiều đã chuẩn hóa) rồi tính điểm bằng

toán tử MaxSim: với mỗi token truy vấn q_i, tìm token tài liệu d_j tương đồng nhất, rồi cộng các cực đại này trên toàn bộ token truy vấn:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} q_i \cdot d_j. \tag{1.2}$$

Vì tương tác giữa hai chuỗi token chỉ diễn ra ở bước cuối sau khi đã mã hóa độc lập, cách tiếp cận này được gọi là *late interaction*. MaxSim cho phép so khớp từng khía cạnh ngữ nghĩa của truy vấn, đổi lại số vector phải lưu tăng hàng chục đến hàng trăm lần so với SVR (MS MARCO: khoảng 600 triệu vector token), và chấm điểm có độ phức tạp $O(N \times M)$ cho mỗi cặp. Hình 1.1 minh họa hai cách biểu diễn, Bảng 1.1 so sánh ba họ phương pháp.



Hình 1.1: Hai cách biểu diễn tài liệu: SVR nén thành một vector rồi lấy tích vô hướng; MVR giữ một vector cho mỗi token rồi lấy MaxSim theo từng token truy vấn và cộng lại.

Bảng 1.1: So sánh ba họ phương pháp truy hồi: BM25, SVR và MVR (ColBERT).

Tiêu chí	BM25	SVR	MVR (ColBERT)
Đơn vị biểu diễn	Từ	1 vector / tài liệu	1 vector / token
Hàm điểm	TF-IDF có chuẩn hóa	Tích vô hướng	MaxSim
Cấu trúc tìm kiếm	Inverted index	ANN (HNSW, IVF)	Clustering + nhiều tầng pruning
Hiểu ngữ nghĩa	Kém	Tốt (mức toàn cục)	Rất tốt (mức token)
Tốc độ	Rất nhanh	Nhanh	Chậm hơn
Kích thước chỉ mục	Nhỏ	Nhỏ	Rất lớn

1.3. Pipeline ba tầng của ColBERTv2/PLAID

Để giảm chi phí của MVR, ColBERTv2 [3] giảm số chiều embedding token (từ 768 xuống 128) rồi clustering bằng K-means với $\sqrt{16 \cdot \text{số token}}$ cụm; mỗi token được lưu dưới dạng chỉ số centroid gần nhất kèm residual quantization 1–2 bit. PLAID [4] tổ chức tìm kiếm thành ba tầng (Hình 1.2):

1. **Tầng I: sinh ứng viên.** Từ mỗi token truy vấn, lấy các centroid gần nhất C_i rồi gom tất cả tài liệu có token thuộc các cụm đó:

$$\mathcal{D}_{\text{cand}} = \bigcup_{q_i \in Q} \bigcup_{c \in C_i} \phi(c), \tag{1.3}$$

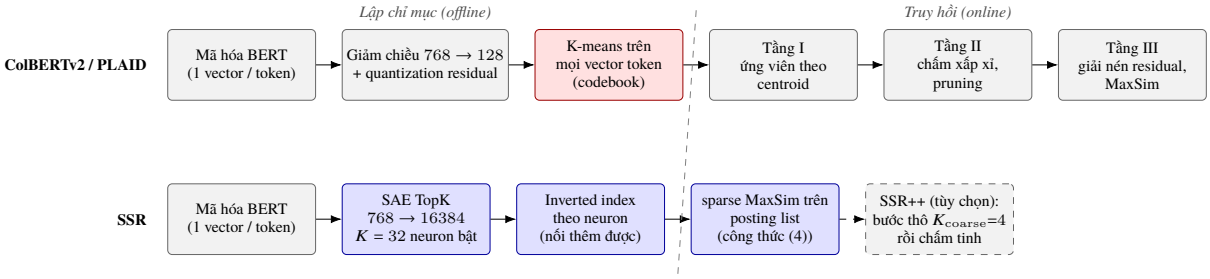
   trong đó $\phi(c)$ là tập tài liệu có token thuộc centroid c. Tầng này loại bỏ hơn 98% kho.

2. **Tầng II: chấm điểm xấp xỉ và pruning.** Dùng centroid thay cho vector thật để ước lượng điểm, giữ lại vài nghìn tài liệu:

$$\hat{S}(Q, D) = \sum_{i=1}^{|Q|} \max_{j=1}^{|D|} q_i \cdot c_{d_j}. \tag{1.4}$$

3. **Tầng III: giải nén và xếp hạng lại.** Khôi phục vector xấp xỉ $\tilde{d}_j \approx c_{d_j} + r_{d_j}$ rồi tính MaxSim đầy đủ trên tập ứng viên nhỏ:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} q_i \cdot \tilde{d}_j. \tag{1.5}$$



Hình 1.2: So sánh pipeline của ColBERTv2/PLAID (ba tầng truy hồi, cần K-means khi lập chỉ mục) và SSR (một giai đoạn trên inverted index). Vẽ lại theo Hình 2 của bài báo [1].

1.4. Hạn chế của hướng tiếp cận dựa trên K-means

Cách tiếp cận trên có ba hạn chế chính, với số liệu do bài báo đưa ra; Bảng 1.2 tổng hợp các hạn chế này cùng cách SSR giải quyết và vị trí trình bày trong báo cáo:

- **Chi phí lập chỉ mục lớn và không cập nhật gia tăng được.** K-means phải chạy trên hàng tỷ vector token, sau đó còn tính residual cho từng token. Trên MS MARCO, ColBERTv2/PLAID mất khoảng 122,9 giờ lập chỉ mục, XTR/WARP khoảng 103,7 giờ, với bộ nhớ đỉnh lần lượt 274,2 GB và 186,7 GB. Khi có tài liệu mới, K-means không thể chèn thêm điểm dữ liệu một cách rẻ, buộc hệ thống phải xây lại chỉ mục.
- **Mất thông tin do nén.** Gom token về centroid cùng mã ngắn loại bỏ đúng phần chi tiết mức token mà MVR muốn giữ, khiến chất lượng xếp hạng bị chặn trần.
- **Pipeline nhiều tầng phức tạp.** Logic điều khiển rối, các lần truy cập bộ nhớ không liên tục và nhiều lượt giải nén làm tăng độ trễ. Bản thân MaxSim vẫn có độ phức tạp $O(N \times M)$, nên độ trễ tăng mạnh với tài liệu dài (ColBERTv2 lên 79,3 ms trên MS MARCO Document).

Bảng 1.2: Hạn chế của phương pháp dựa trên K-means và cách SSR giải quyết, kèm vị trí trình bày trong báo cáo.

Hạn chế	Nguyên nhân	SSR giải quyết bằng	Trình bày ở
Lập chỉ mục chậm, phải xây lại	K-means trên hàng tỷ vector	Inverted index mức neuron, chỉ nối thêm	Chương 3
Mất thông tin	Nén về centroid, giảm chiều	Chiếu lên không gian nhiều chiều sparse	Chương 2
Pipeline phức tạp, $O(N \times M)$	Ba tầng cộng nhiều lượt giải nén	Một giai đoạn; SSR++ pruning coarse-to-fine	Chương 3
Đặc trưng sparse phải đủ phân biệt	Sparse code có thể trùng nhau	Hybrid loss reconstruction + contrastive	Chương 2

1.5. Ý tưởng của SSR và bài báo

Từ các hạn chế trên, bài báo đặt câu hỏi: *liệu có thể giữ được độ chi tiết mức token của MVR, đạt tốc độ tương đương tìm kiếm từ vựng, mà không cần đến K-means clustering hay không?* Ý tưởng của SSR là đảo ngược hướng biến đổi không gian: thay vì nén embedding token xuống không gian ít chiều, SSR chiếu nó lên một không gian *nhiều chiều hơn nhưng cực kỳ sparse* nhờ Sparse Autoencoder (SAE), trong đó mỗi token chỉ kích hoạt $K = 32$ trên $h = 16.384$ neuron. Mỗi neuron hoạt động được coi như một "pseudo-token"; vì số neuron bật rất ít so với số

chiều, mỗi neuron chỉ xuất hiện ở một phần nhỏ token, nên toàn bộ hạ tầng inverted index vốn dùng cho BM25 có thể được tái sử dụng.

Bài báo đóng góp ba điểm: (i) chiếu embedding token lên không gian sparse nhiều chiều để thay thế clustering, cho phép truy hồi ngữ nghĩa đa vector qua một giai đoạn trên inverted index; (ii) hàm mục tiêu hybrid training kết hợp reconstruction và contrastive nhằm vừa giữ thông tin vừa bảo đảm khả năng phân biệt; (iii) inverted index mức neuron cùng thuật toán truy hồi hai bước SSR++, kèm phân tích lý thuyết về sai lệch bị chặn so với điểm dense và đánh giá trên MS MARCO, BEIR và backbone LLM 8B. Bài còn đề xuất hai biến thể: SSR-tok chỉ dùng điểm token sparse và SSR-CLS bổ sung tín hiệu ngữ nghĩa toàn cục từ token `[CLS]`. Theo số liệu công bố, SSR đạt nDCG@10 trung bình 52,9–53,4 trên 13 tập BEIR với độ trễ 17,5–19,5 ms, vừa cao hơn vừa nhanh hơn ColBERTv2, PLAID và SPLADE-v3; các số liệu này được tổng hợp và kiểm chứng ở Chương 4.

Bài báo *No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval* của Lixuan Guo, Yifei Wang, Tiansheng Wen, Aosong Feng, Stefanie Jegelka và Chenyu You được công bố tại ICML 2026 (PMLR 306) [1, 21]. Báo cáo dùng bản arXiv:2605.30120 v3 (03/06/2026) làm bản chính và đối chiếu với bản v4 (28/09/2026) cùng bản PMLR. Mã nguồn được công bố tại https://github.com/Y-Research-SBU/SSR (giấy phép MIT, commit `c64bc05`) [19], checkpoint SSR-tok tại https://huggingface.co/Y-Research-Group/SSR [20].

1.6. Các nghiên cứu liên quan

Tối ưu hóa MVR. ColBERTv2 giảm chiều và nén residual [3]; PLAID [4] và EMVB [8] cải thiện khâu pruning nhiều tầng bằng centroid và bit-vector/SIMD; XTR [6] và WARP [7] đặt mục tiêu truy hồi token rẻ hơn. Tất cả vẫn cần một codebook học bằng K-means và phải dựng lại codebook khi kho thay đổi.

Learned sparse retrieval và inverted index. SPLADE [9] học trọng số sparse trên từ điển 30.522 từ của BERT và truy hồi bằng inverted index, nhưng gộp cả văn bản thành *một* vector sparse trên một từ điển cố định. COIL [5] giữ tương tác mức token nhưng chỉ cho các token khớp từ chính xác. SSR khác hai hướng này ở chỗ không gian sparse được *học* từ SAE, không ràng buộc vào từ điển, và tương tác mức token được giữ cho mọi cặp token chung neuron.

Sparse Autoencoder. SAE với các biến thể TopK [10], BatchTopK, JumpReLU và Gated được dùng nhiều trong nghiên cứu diễn giải mô hình ngôn ngữ, nhằm tách dense embedding thành các "khái niệm" riêng. SSR là một trong những ứng dụng đầu tiên của TopK SAE vào truy hồi: mỗi chiều của sparse code là một neuron khái niệm, không phải một từ, và được dùng làm khóa của inverted index.

1.7. Mục tiêu, câu hỏi kiểm chứng, phạm vi và phương pháp

1.7.1. Mục tiêu và câu hỏi kiểm chứng

Báo cáo có hai mục tiêu. Về lý thuyết, trình bày cơ sở của SSR: biểu diễn sparse và sparse MaxSim, hàm mục tiêu hybrid training, inverted index mức neuron và thuật toán SSR/SSR++. Về thực hành, chạy lại mã nguồn và checkpoint công bố, đối chiếu mã nguồn với từng công thức, kiểm chứng kết quả với baseline đối chứng, và cải tiến mã nguồn. Phần thực hành được tổ chức quanh bốn câu hỏi kiểm chứng:

- **(K1) Tái lập.** Mã nguồn và checkpoint công bố có cho ra chất lượng như Bảng 1 của bài trên BEIR không? Nếu không, sai lệch nằm ở đâu?
- **(K2) Chất lượng so với đối chứng.** Trên cùng pipeline, checkpoint công bố so với ColBERTv2 thế nào trên split test, trên các split train/dev đối chứng của cùng tập dữ liệu, và trên MS MARCO?
- **(K3) Hiệu quả.** Các tuyên bố về thời gian lập chỉ mục, độ trễ, kích thước chỉ mục và chất lượng của SSR++ có quan sát được ở quy mô nhóm đo được không?

- **(K4) Cải tiến.** Có thể tăng tốc truy hồi exact mà giữ nguyên kết quả không, và có phép xấp xỉ nào tốt hơn SSR++ không?

1.7.2. Phạm vi

- *Lý thuyết:* bài báo arXiv:2605.30120 (các bản v3, v4 và bản PMLR 306), cùng các công trình nền ColBERT, ColBERTv2, PLAID, SPLADE, TopK SAE.
- *Thực hành:* mã nguồn chính thức tại commit `c64bc05` và checkpoint SSR-tok trên HuggingFace; các tập NFCorpus, SciFact, FiQA-2018, Quora (BEIR) và MS MARCO passage; baseline đối chứng ColBERTv2 (PLAID) và BM25. Huấn luyện lại từ đầu nằm ngoài phạm vi vì vượt quá tài nguyên của nhóm (một GPU 8 GB); nhóm chỉ sửa và chạy thử đường huấn luyện.

1.7.3. Phương pháp

- **Tổng hợp và phân tích tài liệu:** dịch và đọc bài báo cùng các công trình nền, tính lại các bảng số liệu trong bài và so sánh các phiên bản.
- **Phân tích mã nguồn:** đọc toàn bộ repository, đối chiếu từng hàm với công thức tương ứng, đọc các issue trên GitHub và trả lời của tác giả.
- **Thực nghiệm có đối chứng:** mọi phép đo chạy trên cùng một máy, dùng cùng pipeline cho SSR và ColBERTv2; pipeline được kiểm tra bằng cách tái lập số đã công bố của ColBERTv2 và BM25 trước khi đo SSR; mỗi bản sửa được kiểm chứng bằng ít nhất hai cách tính độc lập.
- **Tối ưu và đo lường:** thiết kế lại kernel chấm điểm, viết test so sánh từng bit với bản gốc, đo độ trễ trung bình, p50, p95 và bộ nhớ trên cùng tập query.

Ở thời điểm thực hiện báo cáo (10/2026), mã nguồn của bài có hai issue công khai từ hai nhóm độc lập báo không tái lập được Bảng 1 [19]; tác giả đã thừa nhận một số lỗi và bản arXiv v4 đã sửa lại cách so sánh trên MS MARCO. Vì vậy, bên cạnh việc tìm hiểu ý tưởng, kiểm tra khả năng tái lập là nội dung trọng tâm của Chương 4.

1.7.4. Cấu trúc báo cáo

Chương 2 trình bày biểu diễn sparse, sparse MaxSim, cận sai lệch so với điểm dense và cách huấn luyện SAE; Chương 3 trình bày inverted index mức neuron và hai thuật toán SSR, SSR++; Chương 4 tổng hợp các kết quả công bố cần kiểm chứng, rồi trình bày môi trường, các sai lệch mã nguồn, kết quả tái lập và so sánh với ColBERTv2/PLAID; Chương 5 trình bày phần cải tiến mã nguồn, thảo luận và kết luận theo từng câu hỏi kiểm chứng. Phụ lục A ghi chi tiết từng lỗi đã sửa kèm đoạn mã trước và sau; Phụ lục B chứng minh hai định lý về cận sai lệch; Phụ lục C tổng hợp các kết quả khác của bài báo; Phụ lục D ghi nguồn của từng số liệu hiệu năng; Phụ lục E hướng dẫn chạy lại. Mã nguồn, patch, script đánh giá và kết quả của nhóm được công bố tại https://github.com/duykhanhxx03/ssr-repro-and-optimization.

CHƯƠNG 2. BIỂU DIỄN SPARSE VÀ HUẤN LUYỆN SSR

Chương này trình bày phần mô hình của SSR: vì sao chuyển từ biểu diễn dense ít chiều sang biểu diễn sparse nhiều chiều, điểm tương đồng được tính như thế nào trên không gian sparse, sai số so với điểm dense bị chặn bởi những đại lượng nào, và SAE được huấn luyện với hàm mục tiêu gì để sparse code vừa giữ thông tin vừa đủ phân biệt. Cuối chương, nhóm đối chiếu thiết lập huấn luyện trong bài với mã nguồn công bố.

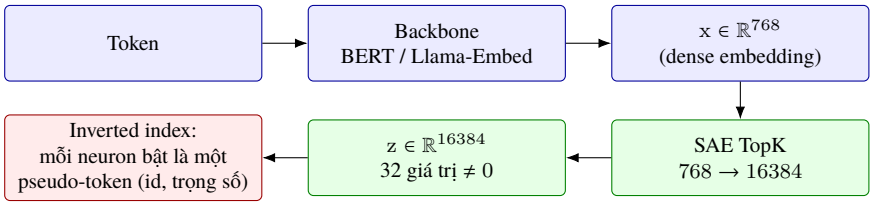
2.1. Từ biểu diễn dense sang biểu diễn sparse

ColBERT chiếu mỗi token về một vector 128 chiều trong đó hầu như mọi chiều đều khác không. Chính đặc điểm dense này khiến inverted index không dùng được: một vector dense "nằm" trên mọi chiều, không có chiều nào đủ đặc trưng để làm khóa tra cứu, nên muốn tìm ứng viên hệ thống phải clustering rồi duyệt theo centroid, tức quay lại nút thắt ở mục 1.4.

Sparse coding [11] biểu diễn một tín hiệu bằng tổ hợp của một số ít phần tử lấy từ một từ điển lớn hơn nhiều so với số chiều tín hiệu (từ điển overcomplete, $h \gg d$). Trong Sparse Autoencoder, mỗi neuron được kỳ vọng đảm nhiệm một "khái niệm" ngữ nghĩa hẹp, và dense embedding của token được xem là superposition của nhiều khái niệm như vậy; nhiệm vụ của SAE là tách các khái niệm đó ra thành các chiều riêng. Với truy hồi, điều quan trọng là hai token gần nghĩa sẽ kích hoạt chung một số neuron, nên một neuron hoạt động đóng vai trò như một pseudo-token, tương tự một từ khóa trong BM25. Vì số neuron bật cho mỗi token rất nhỏ ($K = 32$ trên $h = 16.384$, khoảng 0,2%), mỗi neuron chỉ xuất hiện ở một phần nhỏ token, đủ để xây inverted index. Bảng 2.1 đối chiếu hai cách biểu diễn, Hình 2.1 mô tả luồng biến đổi một token.

Bảng 2.1: So sánh biểu diễn dense của ColBERT và biểu diễn sparse của SSR.

Tiêu chí	Dense (ColBERT)	Sparse (SSR)
Số chiều mỗi token	128	16.384
Số phần tử khác không	128 (100%)	32 (≈0,2%)
Ý nghĩa mỗi chiều	Không diễn giải được	Xấp xỉ một khái niệm
Cấu trúc tìm kiếm	Clustering + ANN	Inverted index
Chi phí tích vô hướng một cặp token	128 phép nhân	≤ 32 (chỉ trên phần giao)



Hình 2.1: Luồng biến đổi một token trong SSR: backbone tạo dense embedding, SAE TopK tạo sparse code 16.384 chiều với 32 giá trị khác không, mỗi chiều hoạt động trở thành một pseudo-token và được ghi vào inverted index.

2.2. TopK Sparse Autoencoder

SAE [10] gồm encoder, toán tử TopK và decoder:

$$z = \mathrm{TopK}\left(W_{\mathrm{enc}}(x - b_{\mathrm{pre}}) + b_{\mathrm{enc}}\right), \tag{2.1}$$

$$\hat{x} = W_{\mathrm{dec}}\, z + b_{\mathrm{pre}}, \tag{2.2}$$

$$\mathcal{L}_{\mathrm{recon}}(k) = \|x - \hat{x}\|_2^2 \quad \text{(khi } z \text{ giữ } k \text{ neuron)}. \tag{2.3}$$

Trong đó $\mathrm{W}_{\mathrm{enc}} \in \mathbb{R}^{h\times d}$, $\mathrm{W}_{\mathrm{dec}} \in \mathbb{R}^{d\times h}$; $\mathrm{b}_{\mathrm{pre}}$ đóng vai trò "tâm" của dữ liệu, được trừ trước khi mã hóa và cộng lại sau khi giải mã. Toán tử TopK giữ K giá trị lớn nhất sau ReLU và đặt phần còn lại bằng không, nên mọi giá trị trong z đều *không âm*; tính chất này được dùng ở mục 2.3 và trong phần tối ưu ở Chương 5. So với SAE dùng phạt L_1, TopK kiểm soát chính xác số neuron hoạt động, không gây co giá trị (shrinkage) làm lệch biên độ sparse code, và không cần dò hệ số phạt. K đồng thời là số pseudo-token của mỗi token, nên quyết định trực tiếp kích thước inverted index ở Chương 3. Hai kỹ thuật kèm theo của TopK SAE cũng được dùng: AuxK (mục 2.6) và annealing K từ $2K$ về K trong giai đoạn đầu huấn luyện.

SAE được áp dụng độc lập cho từng token của truy vấn và tài liệu:

$$Q' = \{\mathrm{z}_{q_1}, \ldots, \mathrm{z}_{q_N}\} \in \mathbb{R}^{h\times N}, \qquad D' = \{\mathrm{z}_{d_1}, \ldots, \mathrm{z}_{d_M}\} \in \mathbb{R}^{h\times M}. \tag{2.4}$$

SSR huấn luyện hai SAE riêng, $\mathbb{E}_{\mathrm{tok}}$ cho các token thường và $\mathbb{E}_{[\mathrm{CLS}]}$ cho token `[CLS]`, với $d = 768$, $h = 16.384$, $K = 32$, $k_{\mathrm{aux}} = 2.048$ khi backbone là BERT-base (Bảng 2.2). Với backbone Llama-Embed-8B, các siêu tham số được điều chỉnh tương ứng: $d = 4.096$, $h = 65.536$, huấn luyện ở định dạng BF16 với $\mathrm{lr} = 2 \cdot 10^{-4}$ và batch toàn cục 32; backbone LLM được đóng băng, còn với BERT-base thì không (mục 2.8).

Bảng 2.2: Siêu tham số huấn luyện SAE với backbone BERT-base theo Bảng 6 của bài báo.

d	h	K	k_{aux}	Optimizer	lr	Batch	Epoch	Warmup	α	β	γ
768	16.384	32	2.048	AdamW	10^{-3}	64	2	20.000	1/32	0,1	0,05

2.3. Late interaction trên không gian sparse

Điểm liên quan trong SSR là một biến thể sparse của MaxSim. Gọi $\mathcal{A}_K(\mathrm{z})$ là tập *chỉ số* của K neuron có giá trị lớn nhất trong z. Với mỗi token truy vấn, thay vì lấy cực đại trên các token tài liệu theo tích vô hướng đầy đủ, ta chỉ cộng trên tập neuron hoạt động chung:

$$S(Q, D) = \sum_{i=1}^{N} \max_{j=1}^{M} \left(\sum_{u \in \mathcal{A}_K(\mathrm{z}_{q_i}) \cap \mathcal{A}_K(\mathrm{z}_{d_j})} z_{q_i}^{(u)}\, z_{d_j}^{(u)} \right). \tag{2.5}$$

Đây là công thức (4) của bài báo. Đọc từ trong ra ngoài: tổng bên trong chỉ chạy trên phần giao các neuron hoạt động của *một cặp* token truy vấn và token tài liệu; phép max chọn token tài liệu khớp nhất cho mỗi token truy vấn; tổng ngoài cùng cộng đóng góp của tất cả token truy vấn. Hai hệ quả theo sau. Nếu phần giao rỗng thì điểm của cặp token bằng không và không cần tính; đây là cơ sở để xây inverted index ở Chương 3. Chi phí cho mỗi cặp token bị chặn bởi K phép nhân thay vì d phép nhân.

Bảng 2.3: Ví dụ tính tay điểm sparse cho token truy vấn q_1 với hai token tài liệu.

Token	**Neuron hoạt động (chỉ số : giá trị)**		
q_1 "neural"	12 : 0,9	305 : 0,7	9001 : 0,2
d_1 "network"	12 : 0,8	77 : 0,5	
d_2 "brain"	305 : 0,6	9001 : 0,4	

Với ví dụ ở Bảng 2.3, token q_1 khớp với d_1 chỉ trên neuron 12 nên đóng góp là $0,9 \times 0,8 = 0,72$; khớp với d_2 trên hai neuron 305 và 9001 nên đóng góp là $0,7 \times 0,6 + 0,2 \times 0,4 = 0,50$. MaxSim lấy giá trị lớn hơn, tức 0,72. Nếu truy vấn có thêm token q_2, tổng $S(Q, D)$ là tổng các cực đại của từng token truy vấn.

Thứ tự các phép toán. Thứ tự trong (2.5) quan trọng và dễ cài sai: với mỗi cặp (token query i, token tài liệu j) phải *cộng* các tích trên neuron chung trước, rồi mới lấy *max* theo j, cuối cùng cộng theo i. Nếu đảo lại, tức lấy max theo từng neuron trên toàn tài liệu rồi mới gộp, điểm của cặp (token query, tài liệu) trở thành $\max_u q_i^{(u)} \max_j d_j^{(u)}$: phép cộng trên các neuron chung bị mất và các neuron không còn bị ràng buộc phải thuộc cùng một token tài liệu.

Ở ví dụ trên, nếu tài liệu gồm cả d_1 và d_2, cách tính sai cho q_1 điểm $\max(0{,}9 \cdot 0{,}8;\ 0{,}7 \cdot 0{,}6;\ 0{,}2 \cdot 0{,}4) = 0{,}72$, trùng với kết quả đúng chỉ vì tình cờ; nếu tài liệu chỉ gồm d_2, nó cho $\max(0{,}42;\ 0{,}08) = 0{,}42$ thay vì $0{,}50$. Mã nguồn công bố của bài báo mắc đúng lỗi này ở chế độ `exact` trên inverted index (mục 4.4).

2.4. Hai biến thể SSR-tok và SSR-CLS

SSR-tok chỉ dùng công thức (2.5) với sparse code của token thường. SSR-CLS cộng thêm tín hiệu ngữ nghĩa toàn cục bằng độ tương đồng cosine giữa sparse code của token `[CLS]` ở truy vấn và tài liệu. Theo bài báo, SSR-CLS cao hơn SSR-tok khoảng 0,5 điểm nDCG@10 nhưng chậm hơn khoảng 2 ms; khi K nhỏ, mô hình phụ thuộc vào `[CLS]` nhiều hơn. Bài báo không nói rõ cách kết hợp điểm `[CLS]` với điểm token, và tác giả chỉ công bố checkpoint SSR-tok, nên toàn bộ thực nghiệm của nhóm ở Chương 4 dùng SSR-tok.

2.5. Cận sai lệch so với điểm dense

Phần này trả lời câu hỏi: khi thay tích vô hướng dense bằng tích vô hướng sparse thì sai số là bao nhiêu. Nội dung dựa trên Phụ lục A của bài báo; chứng minh chi tiết được đưa vào Phụ lục B của báo cáo.

2.5.1. Thiết lập và giả thiết

Gọi $\mathrm{x}, \mathrm{y} \in \mathbb{R}^d$ là hai dense embedding (đã tâm hóa, tức gộp $\mathrm{b}_{\mathrm{pre}}$ vào embedding), $\mathrm{z}_x, \mathrm{z}_y \in \mathbb{R}^h$ là sparse code TopK tương ứng, và $\hat{\mathrm{x}} = \mathrm{W}_{\mathrm{dec}}\mathrm{z}_x$, $\hat{\mathrm{y}} = \mathrm{W}_{\mathrm{dec}}\mathrm{z}_y$ là các vector tái tạo. Hai điểm số được so sánh là

$$S_{\mathrm{dense}}(Q, D) = \sum_{i=1}^{N} \max_j \mathrm{q}_i^{\top}\mathrm{d}_j, \tag{2.6}$$

$$S_{\mathrm{SSR}}(Q, D) = \sum_{i=1}^{N} \max_j \mathrm{z}_{q_i}^{\top}\mathrm{z}_{d_j}, \tag{2.7}$$

trong đó $\mathrm{z}_{q_i}^{\top}\mathrm{z}_{d_j}$ đúng bằng tổng trong ngoặc của (2.5), vì các thành phần ngoài phần giao đều bằng không. Phân tích dựa trên bốn giả thiết ở Bảng 2.4; Giả thiết 3 có nghĩa là với mọi vector sparse a, b có support nằm trong S: $|\mathrm{a}^{\top}(\mathrm{W}_{\mathrm{dec}}^{\top}\mathrm{W}_{\mathrm{dec}} - \mathrm{I})\mathrm{b}| \le \delta\|\mathrm{a}\|_2\|\mathrm{b}\|_2$.

Bảng 2.4: Các giả thiết của phân tích sai lệch bị chặn.

#	Giả thiết	Công thức	Ý nghĩa
1	Reconstruction trung thực	$\|\mathrm{x} - \hat{\mathrm{x}}\|_2 \le \epsilon$, $\|\mathrm{y} - \hat{\mathrm{y}}\|_2 \le \epsilon$	SAE tái tạo tốt embedding gốc
2	Chuẩn embedding bị chặn	$\|\mathrm{x}\|_2 \le B$, $\|\mathrm{y}\|_2 \le B$	Embedding không quá lớn
3	Decoder gần trực giao trên tập hoạt động $S = \mathrm{supp}(\mathrm{z}_x) \cup \mathrm{supp}(\mathrm{z}_y)$	$\big\|(\mathrm{W}_{\mathrm{dec}}^{\top}\mathrm{W}_{\mathrm{dec}} - \mathrm{I})\big\|_{S\to S} \le \delta$	Các cột decoder của neuron đang bật gần như vuông góc, tức mỗi neuron là một hướng độc lập
4	Chuẩn sparse code bị chặn	$\|\mathrm{z}_x\|_2 \le C$, $\|\mathrm{z}_y\|_2 \le C$	Dùng để thu được cận đều theo mọi token

2.5.2. Hai định lý

Định lý A. *Với các Giả thiết 1–3, tích vô hướng của hai dense embedding và tích vô hướng của hai sparse code tương ứng thỏa mãn*

$$\left|\mathrm{x}^{\top}\mathrm{y} - \mathrm{z}_x^{\top}\mathrm{z}_y\right| \le 2B\epsilon + \epsilon^2 + \delta\|\mathrm{z}_x\|_2\|\mathrm{z}_y\|_2. \tag{2.8}$$

Nếu Giả thiết 4 cũng đúng thì vế phải không vượt quá $2B\epsilon + \epsilon^2 + \delta C^2$.

Vế phải của (2.8) gồm hai thành phần: sai số reconstruction của SAE ($2B\epsilon + \epsilon^2$) và sai số hình học của decoder ($\delta\|z_x\|\|z_y\|$), tương ứng với hai bước của chứng minh.

Định lý B. *Giả sử các Giả thiết 1–4 đúng với mọi cặp token truy vấn và token tài liệu* (q_i, d_j) *và đặt* $\eta = 2B\epsilon + \epsilon^2 + \delta C^2$. *Khi đó*

$$\left|S_{\text{dense}}(Q, D) - S_{\text{SSR}}(Q, D)\right| \leq N\eta. \tag{2.9}$$

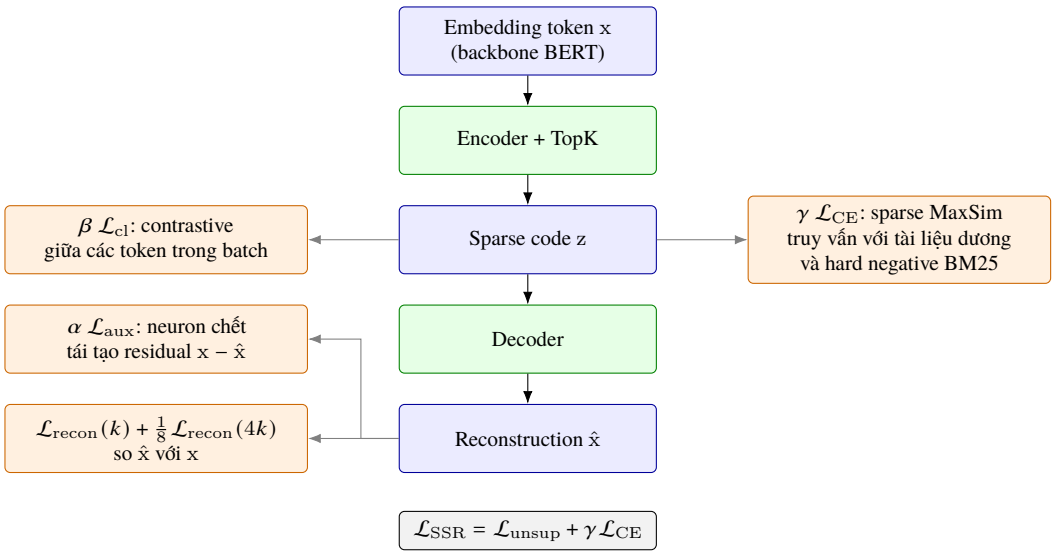
2.5.3. Ý nghĩa và điều kiện áp dụng

Cận (2.9) nói rằng sai lệch điểm số bị kiểm soát bởi ba đại lượng có thể tối ưu trong huấn luyện: ϵ nhỏ nhờ reconstruction loss, δ nhỏ nhờ các neuron tách biệt, và C bị chặn nhờ TopK. Đây là lý do hàm mục tiêu ở mục 2.6 cần thành phần reconstruction bên cạnh thành phần xếp hạng.

Khi đọc cận này cần chú ý vài điểm. Cận tăng tuyến tính theo độ dài truy vấn N; đây là cận trên cho sai lệch *điểm số*, không phải khẳng định về chất lượng xếp hạng với truy vấn dài, vì khoảng cách điểm giữa các tài liệu cũng thay đổi theo N. Với bài toán xếp hạng, điều cần bảo toàn là *thứ tự*: nếu điểm của mỗi tài liệu sai lệch tối đa $N\eta$, thì hai tài liệu có điểm dense cách nhau hơn $2N\eta$ sẽ giữ nguyên thứ tự khi chấm bằng điểm sparse. Đây là điều kiện đủ, không phải điều kiện cần, và các cặp tài liệu có điểm dense gần nhau có thể đổi chỗ. Bài báo không đo ϵ và δ của SAE đã huấn luyện, nên cận chưa được kiểm chứng định lượng và có thể khá lỏng so với thực tế. Thực nghiệm của nhóm trên checkpoint công bố cho thấy tích vô hướng giữa các token sparse khá nhỏ (ngưỡng để lọt top-100 chỉ khoảng 0,22–0,32 cho mỗi token truy vấn), nên các cận trên dùng cho pruning cũng rất lỏng; mục 5.5 phân tích kỹ hơn.

2.6. Hàm mục tiêu hybrid training

Sparse projector phải thỏa mãn hai yêu cầu có phần đối lập. Yêu cầu *reconstruction*: sparse code giữ đủ thông tin của embedding gốc để ϵ nhỏ. Yêu cầu *phân biệt*: sparse code đủ đặc trưng để xếp hạng tài liệu liên quan cao hơn tài liệu không liên quan, điều mà chỉ tối ưu reconstruction không bảo đảm, bởi hai embedding hoàn toàn có thể được tái tạo tốt mà không chia sẻ neuron nào. Vì vậy quá trình huấn luyện là *hybrid training*: vừa không giám sát (reconstruction) vừa có giám sát (xếp hạng). Hình 2.2 tổng quan quá trình này.



Hình 2.2: Tổng quan quá trình huấn luyện SAE trong SSR: embedding token từ backbone đi qua encoder và toán tử TopK để tạo sparse code, sau đó được giải mã; hàm mục tiêu gồm bốn nhóm thành phần: reconstruction, chống neuron chết, contrastive giữa các token và contrastive giữa truy vấn và tài liệu.

2.6.1. Hàm loss không giám sát

Hàm loss không giám sát gồm bốn thành phần (công thức (7) của bài):

$$\mathcal{L}_{\text{unsup}} = \mathcal{L}_{\text{recon}}(k) + \tfrac{1}{8}\,\mathcal{L}_{\text{recon}}(4k) + \alpha\,\mathcal{L}_{\text{aux}}(k_{\text{aux}}) + \beta\,\mathcal{L}_{\text{cl}}. \tag{2.10}$$

Reconstruction loss $\mathcal{L}_{\text{recon}}(k)$ buộc SAE giữ thông tin của embedding gốc khi chỉ cho phép k neuron hoạt động, trực tiếp làm giảm ϵ trong Định lý A.

Multi-TopK $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ yêu cầu SAE tái tạo tốt cả khi giữ $4K = 128$ neuron. Nhờ vậy các neuron ngoài top-K vẫn được học, và có thể thay đổi K lúc suy luận mà không phải huấn luyện lại; đây là nền tảng cho các thí nghiệm K thích ứng của bài (Phụ lục C). Hệ số $\frac{1}{8}$ giữ cho thành phần này không lấn át số hạng reconstruction chính.

Auxiliary loss $\mathcal{L}_{\text{aux}}(k_{\text{aux}})$ chống neuron chết (dead latent), tức neuron lâu ngày không lọt vào TopK nên không bao giờ được cập nhật. Thành phần này chọn $k_{\text{aux}} = 2.048$ neuron "chết lâu nhất" để tái tạo residual $\mathrm{e} = \mathrm{x} - \hat{\mathrm{x}}$, buộc chúng học thông tin hữu ích. Với SSR, hậu quả của neuron chết mang tính hệ thống: neuron không bao giờ hoạt động có posting list rỗng, còn neuron quá phổ biến có posting list quá dài và làm chậm truy vấn; $\mathcal{L}_{\text{aux}}$ vì vậy còn giúp cân bằng chỉ mục.

Sparse contrastive loss $\mathcal{L}_{\text{cl}}$

$$\mathcal{L}_{\text{cl}} = -\frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\log\frac{e^{z_i^\top z_i}}{e^{z_i^\top z_i} + \sum_{j\neq i} e^{z_i^\top z_j}}, \tag{2.11}$$

trong đó $\mathcal{B}$ là tập mọi token trong batch. Thành phần này đẩy sparse code của các token khác nhau ra xa nhau, tránh hiện tượng mọi token cùng bật một nhóm neuron (representation collapse); phân bố neuron đều hơn cũng làm các posting list cân bằng về độ dài.

2.6.2. Supervised contrastive loss $\mathcal{L}_{\text{CE}}$ và hàm mục tiêu tổng

$$\mathcal{L}_{\text{CE}} = -\log\frac{\exp\big(\text{sim}(Q, D^+)/\tau\big)}{\sum_{D\in\mathcal{D}_{\mathcal{B}}}\exp\big(\text{sim}(Q, D)/\tau\big)}. \tag{2.12}$$

Mỗi batch $\mathcal{D}_{\mathcal{B}}$ gồm một tài liệu dương D^+ và các tài liệu âm; tài liệu âm là *hard negative* lấy bằng BM25 trên MS MARCO, tức những tài liệu khớp từ vựng nhưng sai về ngữ nghĩa, buộc mô hình học khác biệt tinh tế. Hàm $\text{sim}(\cdot)$ là sparse MaxSim theo (2.5) với $\mathbb{E}_{\text{tok}}$ và là độ tương đồng cosine với $\mathbb{E}_{[\text{CLS}]}$. $\mathcal{L}_{\text{CE}}$ đưa tín hiệu "truy vấn nào khớp tài liệu nào" vào sparse code, điều mà reconstruction không thể bảo đảm. Công thức (9) trong bài không có nhiệt độ τ; theo tác giả trả lời trên GitHub, giá trị thực dùng là $\tau = 0{,}3$ (mục 2.8). Hàm mục tiêu tổng là

$$\mathcal{L}_{\text{SSR}} = \mathcal{L}_{\text{unsup}} + \gamma\,\mathcal{L}_{\text{CE}}, \qquad \gamma = 0{,}05. \tag{2.13}$$

Bảng 2.5 tổng hợp vai trò của từng thành phần.

Bảng 2.5: Các thành phần của hàm loss và vai trò của từng thành phần.

Thành phần	Loại	Trọng số	Tác động chính	Liên hệ lý thuyết / hệ thống
$\mathcal{L}_{\text{recon}}(k)$	Không giám sát	1	Giữ thông tin	ϵ nhỏ (Định lý A)
$\mathcal{L}_{\text{recon}}(4k)$	Không giám sát	1/8	Linh hoạt K	K thích ứng lúc suy luận
$\mathcal{L}_{\text{aux}}$	Không giám sát	$\alpha = 1/32$	Chống neuron chết	Posting list cân bằng
$\mathcal{L}_{\text{cl}}$	Không giám sát	$\beta = 0{,}1$	Đa dạng sparse code	Tránh representation collapse
$\mathcal{L}_{\text{CE}}$	Có giám sát	$\gamma = 0{,}05$	Khả năng xếp hạng	Mục tiêu truy hồi

2.7. Vai trò của từng thành phần theo ablation của bài báo

Bảng 2.6 là kết quả ablation của bài báo với $K = 32$, đo nDCG@10 trung bình trên BEIR. Đường cơ sở chỉ có reconstruction đạt 46,4 (SSR-tok) và 46,7 (SSR-CLS). Thêm $\mathcal{L}_{\text{aux}}$ tăng lần lượt 1,1 và 1,9 điểm; thêm $\mathcal{L}_{\text{cl}}$ tăng tiếp 0,7 và 0,9 điểm; thêm $\mathcal{L}_{\text{CE}}$ đưa điểm lên 52,9 và 53,4. $\mathcal{L}_{\text{CE}}$ là thành phần đóng góp lớn nhất, khoảng 4,7 điểm cho SSR-tok và 3,9 điểm cho SSR-CLS.

Bảng 2.6: Ablation từng thành phần của hàm loss theo bài báo (nDCG@10 trung bình BEIR, $K = 32$).

α	β	γ	**SSR-tok**	**SSR-CLS**
0	0	0	46,4	46,7
1/32	0	0	47,5	48,6
1/32	0,1	0	48,2	49,5
1/32	0,1	0,05	**52,9**	**53,4**

Khảo sát độ nhạy của bài (Bảng C.9, Phụ lục C) cho thấy γ là trọng số nhạy nhất: tăng lên 0,2 làm chất lượng giảm khoảng 5 điểm, vì $\mathcal{L}_{\text{CE}}$ lấn át reconstruction và ϵ tăng, khớp với Định lý A; α và β ít nhạy hơn, dao động không quá hai điểm. Bài báo chưa làm ablation riêng cho $\mathcal{L}_{\text{recon}}(4k)$, trong khi đây lại là thành phần bị thiếu trong mã nguồn công bố. Nhóm giữ nguyên cấu hình Bảng 2.2 khi đọc checkpoint và không khảo sát lại các trọng số vì không huấn luyện lại.

2.8. Đối chiếu thiết lập huấn luyện với mã nguồn công bố

Khi đối chiếu phần huấn luyện của repository với Bảng 6 và các công thức (7)–(10), nhóm thấy nhiều điểm lệch. Hai nhóm độc lập đã mở issue trên GitHub [19] sau khi huấn luyện lại theo repo chỉ đạt nDCG@10 1,7–2,4 trên NFCorpus (so với 38,8 trong bài). Trong trả lời ngày 31/07/2026 và 04/08/2026, tác giả xác nhận mã công bố nạp SAE khởi tạo ngẫu nhiên, chỉ dùng một hard negative mỗi truy vấn thay vì 32, và thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$; tác giả cũng cho biết một số giá trị thực dùng khác Bảng 6, rằng BERT không được đóng băng, và rằng các token `[MASK]` mở rộng query bị loại trước MaxSim (khớp với `do_query_expansion: false` trong config của checkpoint). Bảng 2.7 đối chiếu ba nguồn.

Bảng 2.7: Thiết lập huấn luyện: bài báo, mặc định của mã nguồn công bố, và trả lời của tác giả trong issue. Cột cuối là cách xử lý của nhóm.

Thiết lập	**Bài báo (Bảng 6)**	**Mã nguồn công bố**	**Tác giả nói trong issue**	**Nhóm xử lý**
Số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$	Có	Tính trong forward nhưng không đưa vào loss	Có dùng, chỉ sót khi công bố	Thêm lại (hệ số 0,125)
γ ($\mathcal{L}_{\text{CE}}$)	0,05	0 (tắt)	0,05	Preset đặt 0,05
Nhiệt độ τ	Không ghi	Không có	0,3	Thêm tham số
Hard negative mỗi query	Không ghi	1	32, gom cùng batch (2 query × 32 triplet = 64)	Thêm `--negatives-per-query` và batch sampler gom theo query
Learning rate	10^{-3}	$2 \cdot 10^{-5}$	10^{-3} "không ổn định", khoảng 10^{-6} tốt hơn	Hai preset: theo Bảng 6 và theo issue
Batch / epoch / warmup	64 / 2 / 20.000	256 / 1 / 0	64 / 2 / 20.000 (≈4% của 2 epoch)	Thêm `--warmup-steps`; preset
k_{aux}	2.048	1.024	–	Preset đặt 2.048
BERT đóng băng?	Không ghi	Không đóng băng	Không đóng băng	Giữ nguyên
Nạp checkpoint	–	Tạo SAE ngẫu nhiên 3.072 latent	Thừa nhận lỗi	Viết lại hàm nạp

Nhóm đã sửa các điểm trên trong bản patch (chi tiết ở Phụ lục A): thêm thành phần `reconstruction_4k`, tham số warmup và số hard negative, một `QueryGroupedBatchSampler` gom các triplet của cùng truy vấn vào một batch như tác giả mô tả, và hai script preset `train_table6.sh` (đúng Bảng 6) và `train_issue1_setup.sh` (theo trả lời của tác giả). Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ (batch 16, 8 hard negative mỗi truy vấn; log cho thấy số hạng $4k$ và warmup hoạt động, checkpoint nạp lại được). Huấn luyện đầy đủ cần khoảng

$32 \times 500\text{k}/64 \approx 250\text{k}$ bước mỗi epoch với batch 64, vượt quá GPU 8 GB của nhóm nên không thực hiện.

Hệ quả cho phần thực nghiệm: checkpoint SSR-tok công bố không thể được tạo ra từ mã nguồn công bố ở trạng thái hiện tại, và một số thiết lập thật (lr, τ, số negative) chỉ được biết qua issue. Mọi đánh giá ở Chương 4 vì vậy dùng checkpoint công bố, và các kết luận về chất lượng chỉ áp dụng cho checkpoint đó.

CHƯƠNG 3. LẬP CHỈ MỤC VÀ TRUY HỒI

Chương này trình bày cấu trúc inverted index mức neuron, hai thuật toán truy hồi SSR và SSR++, phân tích độ phức tạp, và những điểm cần chú ý khi cài đặt mà nhóm rút ra từ việc đối chiếu mã nguồn với bài báo. Các số liệu hiệu quả do bài báo công bố được tổng hợp ở Chương 4 cùng với kết quả nhóm đo, để tách mô tả cơ chế khỏi kết quả đo.

3.1. Inverted index mức neuron

Trong lexical retrieval, mỗi từ có một posting list gồm các cặp (docID, trọng số) và truy vấn chỉ duyệt danh sách của những từ xuất hiện trong truy vấn; Block-Max WAND lưu thêm cận trên của từng block để bỏ qua các tài liệu không thể lọt vào top-k. SSR dùng đúng cấu trúc này với neuron thay cho từ. Gọi z_t là sparse code của token t trong tài liệu D. Có hai mức posting cần phân biệt:

- **Posting mức token** $(D, t, z_t^{(u)})$ cho mọi token t của D có $u \in \mathcal{A}_K(\mathrm{z}_t)$. Đây là thông tin cần để tính đúng công thức (2.5), vì phép max phải chạy theo từng token tài liệu.
- **Posting mức tài liệu** với giá trị tóm tắt

$$\mu_{D,u} = \max_{t \in D} z_t^{(u)}, \qquad \mathcal{I}_u = \{(D, \mu_{D,u}) \mid \mu_{D,u} > 0\}, \tag{3.1}$$

  tức mỗi tài liệu xuất hiện tối đa một lần trong danh sách của neuron u. Bước thô của SSR++ chạy trên mức này.

Mỗi danh sách $\mathcal{I}_u$ được chia thành các block B kích thước cố định (64 phần tử theo Phụ lục D.1 của bài), và mỗi block lưu cận trên

$$U_B = \max_{D \in B} \mu_{D,u}. \tag{3.2}$$

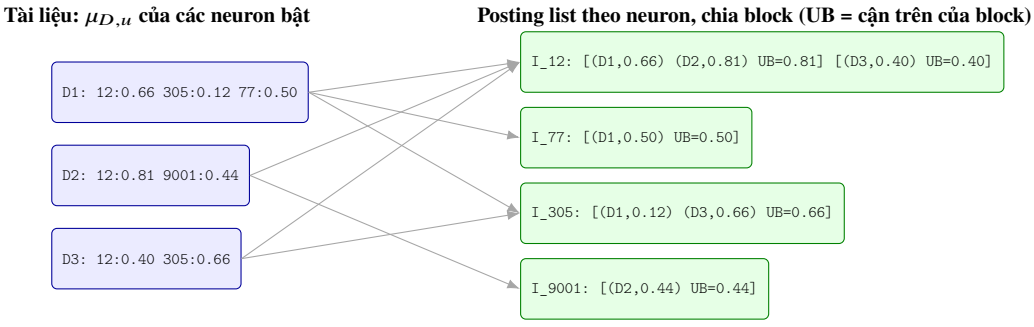
Tổng cộng có $h = 16.384$ danh sách. Thuật toán 3.1 mô tả quy trình lập chỉ mục, Hình 3.1 minh họa cấu trúc mức tài liệu.

Thuật toán 3.1 Xây inverted index mức neuron

Đầu vào: corpus $\mathcal{D}$, backbone, SAE $\mathbb{E}_{\mathrm{tok}}$, kích thước block $b = 64$
Đầu ra: posting mức token, các danh sách $\{\mathcal{I}_u\}$ và cận trên $\{U_B\}$

for mỗi tài liệu $D \in \mathcal{D}$ **do**
  $X \leftarrow \mathrm{backbone}(D)$ ▷ embedding mọi token
  $Z \leftarrow \mathrm{TopK}(\mathbb{E}_{\mathrm{tok}}(X))$ ▷ sparse code, $K = 32$ neuron mỗi token
  for mỗi token t và mỗi neuron $u \in \mathcal{A}_K(\mathrm{z}_t)$ **do**
    thêm $(D, t, z_t^{(u)})$ vào posting mức token của u
  end for
  for mỗi neuron $u \in \bigcup_{t \in D} \mathcal{A}_K(\mathrm{z}_t)$ **do**
    $\mu \leftarrow \max_{t \in D} z_t^{(u)}$; thêm (D, μ) vào $\mathcal{I}_u$
  end for
end for
for $u = 1, \ldots, h$ **do**
  chia $\mathcal{I}_u$ thành các block b phần tử; lưu $U_B = \max \mu$ trong mỗi block
end for

Forward index. Giá trị $\mu_{D,u}$ chỉ là bản tóm tắt theo tài liệu. Để tính đúng (2.5), hệ thống cần giá trị theo *từng token*: hoặc posting mức token như trên, hoặc một forward index lưu sparse code đầy đủ của từng token để chấm lại một tập tài liệu bất kỳ. Nếu chỉ lưu μ thì bước tinh của SSR++ chỉ còn là xấp xỉ. Mã nguồn công bố có lưu posting



Hình 3.1: Cấu trúc inverted index mức neuron (mức tài liệu): mỗi tài liệu đóng góp giá trị $\mu_{D,u}$ vào posting list của từng neuron mà nó kích hoạt; mỗi posting list chia thành block và mỗi block lưu cận trên U_B.

mức token, nhưng phần chấm điểm lại dùng chúng như posting mức tài liệu (mục 3.5). Theo bài báo, bỏ K-means giúp lập chỉ mục MS MARCO trong 7,3 giờ thay vì 122,9 giờ của PLAID với bộ nhớ đỉnh thấp hơn nhiều; các số liệu này được tổng hợp ở Bảng 4.3 và kiểm chứng ở mục 4.7.

3.2. Truy hồi một giai đoạn (SSR)

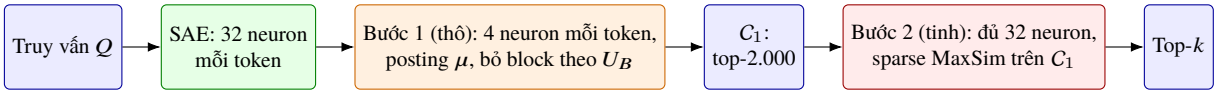
Với mỗi token truy vấn q_i, hệ thống lấy tập $K = 32$ neuron hoạt động $\mathcal{A}_K(\mathrm{q}_i)$, duyệt hợp của $N \times K$ posting list mức token, tích lũy điểm theo từng cặp (tài liệu, token tài liệu), lấy max theo token rồi cộng theo token truy vấn theo đúng (2.5). Ưu điểm là chính xác và chỉ cần một giai đoạn. Nhược điểm là $N \times K$ danh sách vẫn khá nhiều: trên MS MARCO, theo bài báo SSR chạm tới khoảng 54.278 ứng viên và mất 38,6 ms mỗi truy vấn.

3.3. Truy hồi coarse-to-fine (SSR++)

SSR++ giảm chi phí bằng cách chia truy hồi thành hai bước (Thuật toán 3.2, Hình 3.2). Bước thô (coarse scoring) chấm điểm xấp xỉ trên posting mức tài liệu:

$$\hat{S}_{\mathrm{coarse}}(Q, D) = \sum_{i=1}^{N} \sum_{u \in \mathcal{A}_{K_{\mathrm{coarse}}}(\mathrm{q}_i)} q_i^{(u)}\, \mu_{D,u}, \tag{3.3}$$

chỉ dùng $K_{\mathrm{coarse}} = 4$ neuron lớn nhất của mỗi token truy vấn. Kết quả là tập ứng viên C_1 gồm 2.000 tài liệu có điểm thô cao nhất (Phụ lục D.1 của bài). Bước tinh (fine re-scoring) dùng đủ $K = 32$ neuron và tính sparse MaxSim theo (2.5) trên C_1 để có thứ hạng cuối cùng, nên phép tính tốn kém chỉ áp dụng cho một phần nhỏ kho.



Hình 3.2: Sơ đồ SSR++: bước chấm điểm thô với 4 neuron mỗi token và pruning theo cận trên của block, rồi bước tinh chỉnh với đủ 32 neuron trên 2.000 ứng viên.

3.3.1. Điều kiện bỏ block

Thuật toán 3.2 minh họa pipeline; bài báo (Phụ lục D.1) chỉ nói "bỏ qua block khi U_B không đủ để đưa tài liệu vào top-$|C_1|$" mà không viết rõ điều kiện. Để việc bỏ block không làm mất ứng viên của *bước thô*, cần một cận trên cho điểm thô *cuối cùng* của mỗi tài liệu trong block, gồm ba phần: điểm đã tích lũy $\mathrm{score}[D]$, đóng góp của chính cặp (i, u) đang duyệt (tối đa $z_{q_i}^{(u)} U_B$), và đóng góp của mọi cặp (i', u') chưa duyệt (tối đa $\sum z_{q_{i'}}^{(u')} U_{\max}(u')$, với $U_{\max}(u')$ là cận trên toàn danh sách). Riêng $z_{q_i}^{(u)} U_B$ chỉ chặn một phần đóng góp, nên so sánh nó với θ là chưa đủ. Đây là cách làm của Block-Max WAND, và nó đòi hỏi duyệt theo tài liệu (document-at-a-time) hoặc giữ cận

Thuật toán 3.2 SSR++: truy hồi coarse-to-fine

Đầu vào: truy vấn Q, chỉ mục $\{\mathcal{I}_u, U_B\}$, forward index, $K_{\text{coarse}} = 4$, $|C_1| = 2000$, $K = 32$
Đầu ra: top-k tài liệu

$Z_q \leftarrow \text{TopK}(\mathbb{E}_{\text{tok}}(\text{backbone}(Q)))$
score $\leftarrow \{\}$; $\theta \leftarrow 0$ ▷ bước thô; θ là điểm thô thứ $|C_1|$ hiện tại
for $i = 1, \ldots, N$ **do**
  for mỗi u trong K_{coarse} neuron lớn nhất của z_{q_i} **do**
    for mỗi block B trong $\mathcal{I}_u$ **do**
      if cận trên của điểm thô của mọi $D \in B$ (mục 3.3.1) nhỏ hơn θ **then**
        bỏ qua B
      end if
      for mỗi $(D, \mu) \in B$ **do**
        score$[D]$ += $z_{q_i}^{(u)} \cdot \mu$; cập nhật θ
      end for
    end for
  end for
end for
$C_1 \leftarrow$ 2.000 tài liệu có score cao nhất
for mỗi $D \in C_1$ **do**
  $S[D] \leftarrow \text{SparseMaxSim}(Z_q, Z_D)$ ▷ bước tinh, công thức (2.5), dùng forward index
end for
return top-k theo S

cho phần còn lại. Ngay cả khi làm đúng, điều kiện này chỉ bảo đảm C_1 đúng là top-2.000 theo *điểm thô*, không bảo đảm C_1 chứa top-k theo điểm chính xác, vì điểm thô không phải cận của (2.5) (mục 3.3.2).

3.3.2. Tính xấp xỉ của bước thô

Bước thô *không phải* là MaxSim. Nó cộng các đóng góp $\mu_{D,u}$ trên các neuron mà không quan tâm chúng có đến từ cùng một token tài liệu hay không, nên có thể đánh giá cao một tài liệu gồm nhiều token khớp rời rạc. Mặt khác, vì chỉ dùng $K_{\text{coarse}} = 4$ trong $K = 32$ neuron của mỗi token truy vấn, nó bỏ qua đóng góp của 28 neuron còn lại, nên cũng *không phải cận trên* của (2.5) như bài báo gọi ("điểm cận trên xấp xỉ"). Vì vậy bước tinh là bắt buộc, và độ chính xác cuối cùng phụ thuộc vào việc tài liệu đúng có lọt vào C_1 hay không. Bài báo cho rằng mức mất là không đáng kể (Bảng 4.4); kết quả nhóm đo ở mục 4.7 cho thấy trên FiQA 57.638 tài liệu với checkpoint công bố, SSR++ mất 4,6 điểm nDCG@10, còn trên NFCorpus và SciFact không mất vì 2.000 ứng viên phủ gần hết corpus.

3.4. Độ phức tạp

Gọi $\bar{L}_{\text{tok}}$ là độ dài trung bình của posting mức token (số cặp (tài liệu, token) của một neuron) và $\bar{L}_{\text{doc}}$ là độ dài trung bình của $\mathcal{I}_u$ (số tài liệu của một neuron, $\bar{L}_{\text{doc}} \le \bar{L}_{\text{tok}}$). Chi phí duyệt của hai thuật toán là

$$\text{SSR:} \quad O(N \cdot K \cdot \bar{L}_{\text{tok}}), \tag{3.4}$$

$$\text{SSR++ (bước thô):} \quad O(N \cdot K_{\text{coarse}} \cdot L_{\text{skip}}), \qquad K_{\text{coarse}} \ll K,\ L_{\text{skip}} \le \bar{L}_{\text{doc}}, \tag{3.5}$$

trong đó L_{skip} là số phần tử thực sự phải duyệt sau khi bỏ block. Bước tinh của SSR++ tốn thêm $O(|C_1| \cdot N \cdot K)$ trong trường hợp xấu nhất trên forward index. Về bộ nhớ, posting mức token chiếm K phần tử cho mỗi token của corpus, còn posting mức tài liệu chiếm tối đa $K \cdot M$ nhưng thường nhỏ hơn nhiều vì các token trong một tài liệu chia sẻ neuron. Bảng 3.1 so sánh với PLAID.

Độ dài $\bar{L}_{\text{tok}}$ và $\bar{L}_{\text{doc}}$ phụ thuộc vào phân bố kích hoạt neuron. Nếu có neuron quá phổ biến (tương tự từ dừng trong BM25), danh sách của nó rất dài và làm chậm truy vấn; ngược lại neuron quá hiếm làm tăng số chiều lãng phí. Đây là lý do các hàm $\mathcal{L}_{\text{aux}}$ và $\mathcal{L}_{\text{cl}}$ ở mục 2.6 có vai trò hệ thống chứ không chỉ là kỹ thuật huấn luyện.

Bảng 3.1: So sánh độ phức tạp giữa PLAID, SSR và SSR++.

Phương pháp	Sinh ứng viên	Chấm điểm cuối
PLAID	Tìm centroid gần nhất cho N token, gom tài liệu	MaxSim dense $O(N \cdot M \cdot d)$ trên ứng viên sau giải nén
SSR	Duyệt $N \cdot K$ danh sách mức token, độ dài trung bình $\bar{L}_{\text{tok}}$	Gộp max/sum ngay khi duyệt; chi phí mỗi cặp token bị chặn bởi K
SSR++	Duyệt $N \cdot K_{\text{coarse}}$ danh sách mức tài liệu, bỏ block theo cận trên	sparse MaxSim trên 2.000 ứng viên bằng forward index

3.5. Những điểm cần chú ý khi cài đặt

Đối chiếu mã nguồn công bố với các công thức trong chương này, nhóm rút ra bốn điểm mà người cài đặt dễ mắc lỗi; mã nguồn của tác giả mắc cả bốn (chi tiết ở mục 4.4 và Phụ lục A):

1. **Thứ tự cộng rồi max trong** (2.5)**.** Bộ cộng dồn trên inverted index phải gom theo hàng (tài liệu, token tài liệu) rồi mới lấy max theo token; nếu lấy max theo từng neuron trên toàn tài liệu rồi gộp, kết quả là $\sum_i \max_u q_i^{(u)} \mu_{D,u}$ chứ không phải MaxSim (mục 2.3).

2. **Hệ chỉ số neuron của truy vấn và chỉ mục.** Khi chỉ mục nén các neuron thực sự xuất hiện trong corpus thành chỉ số liên tiếp, truy vấn phải được ánh xạ sang cùng hệ chỉ số; quên bước này làm phần giao neuron gần như rỗng.

3. **Bước thô của SSR++.** Phải cài đúng (3.3) trên posting μ với $K_{\text{coarse}} = 4$, block 64, 2.000 ứng viên toàn cục; tham số khác sẽ cho một thuật toán khác với bài.

4. **Bộ nhớ khi dựng chỉ mục.** Với hàng trăm triệu posting (FiQA: 258 triệu), không thể tạo một object Python cho mỗi posting; cần đếm và điền bằng mảng numpy hoặc kernel biên dịch.

CHƯƠNG 4. TÁI LẬP VÀ KIỂM CHỨNG

Chương này là phần thực hành của nhóm. Mục 4.1 tổng hợp những kết quả công bố liên quan trực tiếp đến các câu hỏi kiểm chứng; các kết quả khác của bài được đưa vào Phụ lục C. Các mục sau trình bày môi trường và giao thức đo, việc kiểm tra pipeline bằng baseline, các sai lệch trong mã nguồn, kết quả trên split test và split đối chứng, MS MARCO, và so sánh hiệu năng với ColBERTv2/PLAID. Số liệu của tác giả được ghi kèm "theo bài báo", số liệu do nhóm tự chạy được ghi kèm "nhóm đo". Bảng 4.1 tóm tắt bốn câu hỏi của mục 1.7, phép kiểm tra tương ứng và kết quả chính; Phụ lục D ghi nguồn (file kết quả, tập query, số luồng, thiết bị) của từng số liệu hiệu năng.

Bảng 4.1: Các câu hỏi kiểm chứng, phép kiểm tra, kết quả chính và giới hạn.

Câu hỏi	Phép kiểm tra	Kết quả chính	Giới hạn
(K1) Tái lập	Chạy mã gốc và mã đã sửa với checkpoint công bố trên 4 tập BEIR; đối chiếu từng hàm với công thức	Mã gốc cho 8,3 / 7,5 nDCG@10 (NFCorpus / SciFact); sau sửa 43,97 / 87,83. 10 điểm sai lệch, 9 đã xử lý	Chưa huấn luyện lại; `token_skip_mode` chưa triển khai
(K2) Chất lượng so với đối chứng	Cùng pipeline với ColBERTv2 trên split test và split train/dev đối chứng; MS MARCO re-ranking BM25 top-100	Vượt ColBERTv2 trên split test của 3 tập; kém 0,7–12,6 điểm trên split đối chứng; MS MARCO MRR@10 23,18 so với 35,86	Nguyên nhân chênh lệch theo split chưa xác định; MS MARCO chỉ re-ranking
(K3) Hiệu quả	FiQA 57.638 tài liệu, cùng máy, so với PLAID (fast-plaid, GPU)	Mã hóa chiếm phần lớn thời gian lập chỉ mục; chỉ mục SSR lớn hơn PLAID 4–7 lần; SSR++ mất 4,6 điểm	Một corpus, một máy; SSR trên CPU, PLAID trên GPU
(K4) Cải tiến	Kernel mới, test từng bit, đo trên cùng 100 query (Chương 5)	Nhanh hơn 29,6 lần (12 luồng) so với exact của repo; GPU tương đương; LB-rerank trùng 99,7% top-10 với 0,67 thời gian	Chỉ mục lớn hơn khoảng 1,6 lần so với repo

4.1. Những kết quả công bố cần kiểm chứng

Bài báo đánh giá trên 13 tập BEIR [12] (zero-shot) và MS MARCO passage [13] (in-domain), so với các hệ dense MVR (ColBERT, ColBERTv2, PLAID, COIL, AligneR, CITADEL, XTR) và các hệ learned sparse retrieval (SPLADE-v2, SPLADE-v3), tất cả huấn luyện trên MS MARCO với hard negative từ BM25. Cấu hình SSR dùng backbone BERT-base-uncased, $K = 32$, $h = 16.384$, độ dài truy vấn tối đa 32; SSR++ dùng block 64, $K_{\text{coarse}} = 4$ và 2.000 ứng viên. Độ trễ được đo bằng ms/truy vấn trên GPU A100 40 GB sau 1.000 truy vấn khởi động; trừ khi ghi rõ, độ trễ của SSR-tok và SSR-CLS trong các bảng là của SSR++. Danh sách tập dữ liệu và các kết quả về LLM backbone, LoTTE, LIMIT, độ nhạy h/K nằm ở Phụ lục C.

4.1.1. Chất lượng trên BEIR và MS MARCO

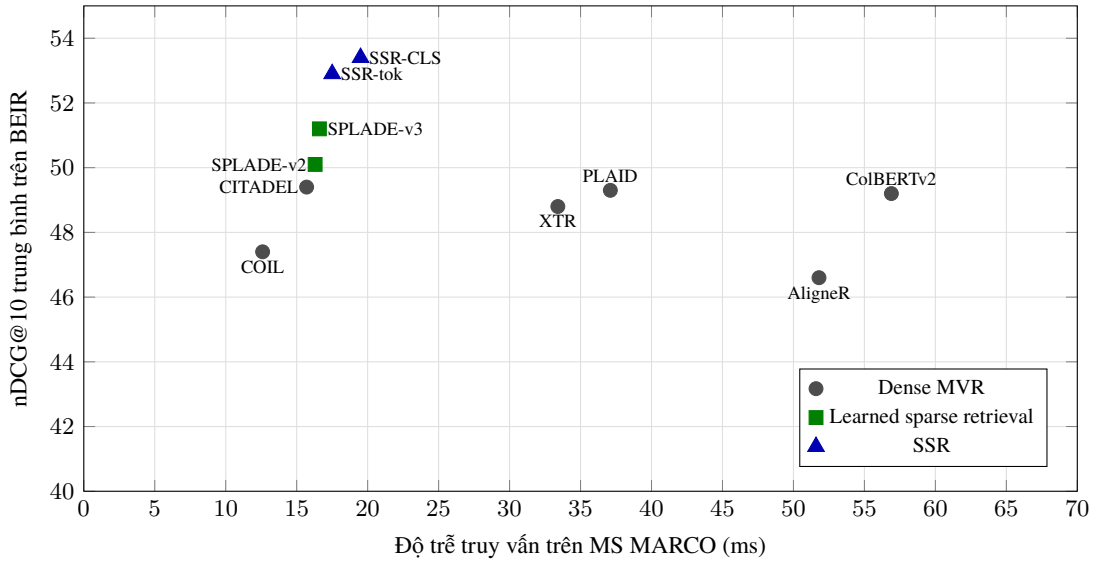
Bảng 4.2 trích từ Bảng 1 của bài (bản arXiv v3) các phương pháp và tập dữ liệu liên quan đến thực nghiệm của nhóm; bảng đầy đủ 13 tập là Bảng C.2 ở Phụ lục C. Hình 4.1 vẽ lại toàn bộ Bảng 1 dưới dạng chất lượng so với độ trễ. Theo số liệu công bố, SSR-CLS đạt trung bình cao nhất với 53,4, hơn SPLADE-v3 khoảng 2,2 điểm và hơn PLAID khoảng 4,1 điểm; SSR-tok với 17,5 ms nhanh gần gấp đôi ColBERTv2 và PLAID; SSR đứng đầu trên 9 trong 13 tập. SSR thấp hơn SPLADE-v3 trên ArguAna và thấp hơn SPLADE-v2 trên Touché, hai tập có truy vấn tranh luận dài, nơi khớp từ vựng chính xác quan trọng hơn.

4.1.2. Hiệu quả lập chỉ mục và truy hồi

Bảng 4.3 tổng hợp các tuyên bố của bài về chi phí lập chỉ mục trên MS MARCO. Trên CPU (Bảng C.6, Phụ lục C), SSR-tok lập chỉ mục trong 7,3 giờ so với 122,9 giờ của PLAID, tức khoảng 1/15, với độ trễ truy hồi 49,1 ms so với 156,3 ms; MRR@10 của SSR-tok trên bảng này là 39,5, ngang PLAID (39,6) và EMVB (39,8). Về huấn luyện, bài cho biết SSR-tok giảm khoảng 53% thời gian so với ColBERTv2 (24,2 giờ).

Bảng 4.2: Trích Bảng 1 của bài báo (arXiv v3): nDCG@10 trên các tập BEIR mà nhóm đã đo, cột MS là MS MARCO (xem mục 4.1.4 về độ đo của cột này), độ trễ ms/truy vấn trên MS MARCO, cột TB là trung bình 13 tập BEIR.

Phương pháp	Độ trễ	MS	NF	FQ	SF	QU	TB
ColBERTv2	56,9	39,8	33,5	36,1	69,1	85,3	49,2
PLAID	37,1	39,7	33,8	36,3	69,2	85,0	49,3
SPLADE-v3	16,6	43,9	35,8	37,5	71,0	81,4	51,2
SSR-tok	17,5	45,2	38,8	39,4	72,9	88,3	52,9
SSR-CLS	19,5	45,5	39,1	40,0	73,3	88,7	53,4



Hình 4.1: Chất lượng trung bình trên BEIR so với độ trễ truy vấn của các phương pháp, vẽ lại từ số liệu công bố ở Bảng 1 của bài báo (arXiv v3). Theo số liệu này, hai biến thể SSR nằm ở góc trên bên trái, tức vừa nhanh vừa chính xác hơn các baseline.

4.1.3. Hiệu quả của SSR++

Theo Bảng 4.4, SSR++ giảm số ứng viên khoảng 17 lần và độ trễ khoảng 2,2 lần trên MS MARCO, trong khi MRR@10 chỉ giảm 0,1 điểm; bài định vị SSR++ cho các kho dữ liệu động. Số ứng viên 3.196 lớn hơn $|C_1| = 2.000$ của Phụ lục D.1, nên cấu hình dùng cho Bảng 5 có thể khác cấu hình được mô tả.

4.1.4. Các phiên bản của bài báo và vấn đề độ đo

Khi tính lại các bảng, nhóm thấy cột MS MARCO của Bảng 1 trộn hai độ đo. Bảng 4.5 tóm tắt theo từng phiên bản. Hệ quả là lợi thế in-domain của SSR trong bản v3 và bản PMLR (+5,4 điểm) phần lớn đến từ việc so nDCG@10 với MRR@10; sau khi v4 đổi mọi baseline sang nDCG@10, lợi thế còn +1,0. Bảng 15 (CPU) ở mọi bản vẫn ghi MRR@10 và cho thấy SSR-tok ngang PLAID trên MS MARCO.

Bảng 4.3: Chi phí lập chỉ mục trên MS MARCO theo bài báo.

Tiêu chí	ColBERTv2/PLAID	XTR/WARP	SSR
Bước tốn kém nhất	K-means + residual	K-means + residual	Mã hóa + ghi posting
Thời gian (CPU)	122,9 h	103,7 h	7,3 h (tok) / 7,8 h (CLS)
Bộ nhớ đỉnh	274,2 GB	186,7 GB	34,6 GB (tok) / 55,1 GB (CLS)
Kích thước chỉ mục	22,1 GB	55,9 GB	18,5 GB
Thêm tài liệu mới	Xây lại	Xây lại	Chỉ nối thêm (append-only)

Bảng 4.4: So sánh SSR và SSR++ trên MS MARCO passage theo bài báo (Bảng 5).

Phương pháp	Số ứng viên	Độ trễ (ms)	MRR@10
SSR	54.278	38,6	45,3
SSR++	3.196	17,5	45,2

Bảng 4.5: Cột MS MARCO của Bảng 1 theo từng phiên bản của bài báo.

Phiên bản	Nhãn độ đo cột MS	SSR-tok	ColBERTv2 / PLAID	Hệ quả so sánh
arXiv v3 (03/06/2026)	"MRR@10" theo chú thích	45,2 (thực ra là nDCG@10)	39,8 / 39,7 (MRR@10)	So lệch chỉ số; lợi thế in-domain +5,4
PMLR 306 (bản chính thức)	nDCG@10	45,2	39,8 / 39,7 (vẫn là MRR@10 của v3)	Lệch chỉ số còn trong bản xuất bản
arXiv v4 (28/09/2026)	nDCG@10	45,2	44,2 / 44,1	Lợi thế còn +1,0; sửa trung bình ColBERT 41,5 → 44,8 và XTR ở Bảng 9 47,6 → 45,0; thí nghiệm theo quy mô đổi từ "SSR giảm 5,3% so với 8,9%" thành "4,2% so với 4,7%"
Bảng 15 (CPU), mọi bản	MRR@10	39,5	PLAID 39,6; EMVB 39,8	SSR-tok ngang PLAID theo MRR@10

4.2. Môi trường, checkpoint, dữ liệu và giao thức đo

Bảng 4.6 và 4.7 mô tả môi trường và các tập dữ liệu. Các tập BEIR tải theo định dạng MTEB bằng script `prepare_mteb_eval.py` của repo, có bổ sung các split train/dev từ bộ BEIR gốc để phục vụ kiểm tra ở mục 4.5.2. Mã nguồn, patch, script đánh giá, kết quả và log nằm tại https://github.com/duykhanhxx03/ssr-repro-and-optimization; cấu trúc repository gốc và lệnh chạy ghi ở Phụ lục E.

Bảng 4.6: Môi trường thực nghiệm của nhóm.

Hạng mục	Giá trị
Phần cứng	CPU Intel i5-12400F (6 nhân / 12 luồng), RAM 32 GB, GPU NVIDIA RTX 5060 8 GB
Hệ điều hành	Windows 11
Phần mềm	Python 3.13, PyTorch 2.11 (CUDA 12.8), transformers 5.3, sentence-transformers 5.3, PyLate 1.6 (ColBERT + fast-plaid), numba 0.61, numpy, scipy
Mã nguồn	`Y-Research-SBU/SSR` commit `c64bc05` (26/06/2026; không có commit nào mới hơn tới ngày 12/10/2026)
Checkpoint	`SSR-tok` tải từ HuggingFace ngày 11/10/2026 (tác giả đưa lên ngày 05/10/2026)
Baseline	`colbert-ir/colbertv2.0` chạy qua PyLate; BM25 cài bằng numpy theo công thức Lucene ($k_1 = 0{,}9$, $b = 0{,}4$)

4.2.1. Độ đo

$$\mathrm{MRR@10} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q} \quad (\text{bằng } 0 \text{ nếu } \mathrm{rank}_q > 10), \tag{4.1}$$

$$\mathrm{DCG@10} = \sum_{i=1}^{10} \frac{2^{\mathrm{rel}_i} - 1}{\log_2(i+1)}, \qquad \mathrm{nDCG@10} = \frac{\mathrm{DCG@10}}{\mathrm{IDCG@10}}, \tag{4.2}$$

trong đó rel_i là mức liên quan của kết quả thứ i và IDCG là DCG của xếp hạng lý tưởng [16]. BEIR dùng mức liên quan có trọng số (NFCorpus có 3 mức, TREC DL có 4 mức), nên nDCG phải tính với $2^{\mathrm{rel}} - 1$ chứ không phải nhãn nhị phân. Mọi điểm nDCG@10 và MRR@10 trong báo cáo được ghi trên thang 0–100.

Bảng 4.7: Các tập dữ liệu nhóm đã dùng.

Tập dữ liệu	Lĩnh vực	Số tài liệu	Query (test)	Split khác đã dùng
NFCorpus	y sinh	3.633	323	dev 324, train 2.590
SciFact	khoa học	5.183	300	train 809
FiQA-2018	tài chính	57.638	648	dev 500, train 1.000 (mẫu)
Quora	câu hỏi trùng	522.931	10.000	dev 5.000
MS MARCO passage	web, in-domain	8.841.823	dev-small 6.980; DL19 43	không

4.2.2. Giao thức đo thời gian và bộ nhớ

Mọi phép đo chạy trên cùng máy ở Bảng 4.6, khi máy không có tác vụ khác. Query được mã hóa trước và lưu lại; đồng hồ chỉ đo bước truy hồi top-100 trên query đã mã hóa, một query mỗi lần, sau 3 query khởi động (JIT của numba), mỗi query đo một lần; báo cáo trung bình, p50 và p95. Với kernel GPU, thời gian tính tới khi kết quả top-k về tới CPU. Với PLAID, query cũng được mã hóa trước rồi gọi `search` với $k = 100$ theo cùng cách. Số luồng CPU (1, 6, 12) do numba điều khiển; số liệu PLAID đo trên GPU vì bản CPU của fast-plaid không dựng xong chỉ mục sau 50 phút. Thời gian "dựng chỉ mục" không gồm mã hóa corpus: với SSR gồm dựng forward index, posting μ và posting token; với PLAID gồm K-means và nén residual. Bộ nhớ RAM là RSS và đỉnh working set của tiến trình theo `psutil`; kích thước chỉ mục là tổng byte của các mảng đã lưu. Mọi đơn vị GB trong báo cáo là 10^9 byte.

4.3. Kiểm tra pipeline bằng baseline và các phép tính độc lập

Trước khi đo SSR, nhóm kiểm tra rằng pipeline đánh giá (đọc dữ liệu, tính nDCG có trọng số, mã hóa, MaxSim) cho đúng số đã công bố với các mô hình đã biết. Bảng 4.8 cho thấy sai lệch dưới 0,6 điểm, nằm trong biên độ khác biệt phiên bản thư viện. Phép kiểm tra này làm giảm khả năng các khác biệt sau này của SSR so với bài báo bắt nguồn từ những thành phần pipeline đã được kiểm tra; nó không loại trừ các nguyên nhân nằm ngoài pipeline, chẳng hạn ở checkpoint hay ở cách mã hóa mà mã nguồn chưa mô tả.

Bảng 4.8: Tái lập số đã công bố của các baseline bằng pipeline của nhóm.

Mô hình	Tập / độ đo	Đã công bố	Nhóm đo
ColBERTv2	SciFact nDCG@10	69,3	69,16
ColBERTv2	NFCorpus nDCG@10	33,8	34,40
ColBERTv2	FiQA nDCG@10	35,6	35,80
ColBERTv2	Quora nDCG@10	85,2	85,67
BM25 (Lucene)	MS MARCO dev-small MRR@10	18,4 (Anserini)	18,38
BM25 (Lucene)	TREC DL19 nDCG@10	50,6 (Anserini)	50,72

Riêng phép chấm điểm sparse MaxSim, phần dễ sai nhất (mục 3.5), được kiểm chứng bằng ba cách tính độc lập: MaxSim trên vector sparse dạng dense bằng PyTorch, một cài đặt bằng `scipy.sparse` viết riêng, và bản "unblocked" của repo sau khi sửa cùng lỗi. Cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact.

4.4. Sai lệch mã nguồn và trạng thái xử lý

Nhóm đối chiếu từng hàm trong repo với công thức tương ứng của bài. Bảng 4.9 tổng hợp các điểm sai lệch cùng trạng thái xử lý. Các điểm 2, 6, 7, 9 đã được tác giả xác nhận trong hai issue trên GitHub (mục 2.8); các điểm 1, 3, 4, 5, 8, 10 là phát hiện của nhóm. Chi tiết từng điểm cùng đoạn mã trước và sau khi sửa được trình bày trong Phụ lục A.

Tổng cộng nhóm phát hiện 10 điểm sai lệch và đã xử lý 9 điểm bằng bản sửa hoặc bổ sung cấu hình (7 điểm sửa mã, 1 điểm thêm preset theo Bảng 6, 1 điểm sửa theo mô tả của tác giả trong issue). Điểm còn lại là cơ chế `token_skip_mode` ghi trong config của checkpoint nhưng không có trong mã nguồn: nhóm không biết tác giả có bỏ

Bảng 4.9: Các sai lệch giữa mã nguồn công bố và bài báo, kèm trạng thái xử lý của nhóm.

#	Sai lệch	Ảnh hưởng	Theo bài	Trạng thái
1	Chế độ `exact` trên inverted index tính $\sum_i \max_u q_i^{(u)} \max_j d_j^{(u)}$ thay vì MaxSim; đường Python, numba và hàm "tham chiếu" sai giống nhau	NFCorpus 8,33 / SciFact 7,46 → 43,97 / 87,83	Công thức (4)	Đã sửa
2	`load_ssr` dựng SAE ngẫu nhiên 3.072 latent thay vì nạp checkpoint; bỏ qua `config_sentence_transformers.json` (độ dài tài liệu 180 thay vì 512, bật query expansion)	Không nạp được mô hình	Không ghi	Đã sửa
3	Đường `e2e-index` bỏ mất `latent_remap`: query và chỉ mục dùng hai hệ chỉ số neuron khác nhau	NFCorpus 1,2 → 44,0	Không ghi	Đã sửa
4	Bước thô SSR++ không theo công thức (12); tham số 8 / 32.768 / 512 thay vì $K_{\text{coarse}} = 4$, 2.000 ứng viên, block 64	SSR++ khác bài	(11)–(12), D.1	Đã sửa
5	nDCG tính với nhãn nhị phân, bỏ mức liên quan	≈1 điểm trên NFCorpus	D.1	Đã sửa
6	Thiếu số hạng $\frac{1}{8}\mathcal{L}_{\text{recon}}(4k)$ trong loss	Huấn luyện	(7)	Đã sửa
7	Tham số huấn luyện mặc định lệch Bảng 6 ($\gamma = 0$ tắt $\mathcal{L}_{\text{CE}}$, lr $2 \cdot 10^{-5}$, batch 256, 1 epoch, không warmup, $k_{\text{aux}} = 1024$)	Huấn luyện	Bảng 6	Thêm preset
8	Config SAE của checkpoint có khóa lạ; dựng chỉ mục tạo một object Python cho mỗi posting (tràn RAM từ ~50k tài liệu)	Không chạy được corpus lớn	Không ghi	Đã sửa
9	Data loader chỉ lấy 1 hard negative mỗi query; tác giả dùng 32 và gom theo query (chỉ nói trong issue #1)	Huấn luyện	Không ghi	Sửa theo issue
10	`token_skip_mode` (bỏ stopword) chỉ có trong config checkpoint, mã nguồn không có	Mã hóa	Không ghi	Chưa triển khai

token nào khi mã hóa hay không, nên cách mã hóa của nhóm có thể khác cách tạo ra số liệu trong bài; đây là nguồn sai lệch còn lại có thể ảnh hưởng đến khả năng tái lập. Toàn bộ bản sửa được đóng gói thành một patch (16 file, áp dụng sạch lên commit `c64bc05`) kèm 7 unit test trong `SSR/tests` và 9 test cho kernel tối ưu.

Trong các điểm trên, điểm 1 ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue trên GitHub. Bộ cộng dồn của chế độ `exact` lấy giá trị lớn nhất theo từng neuron rồi mới gộp, nên điểm của cặp (token query i, tài liệu D) trở thành $\max_u q_i^{(u)} \mu_{D,u}$ thay vì MaxSim theo (2.5), đúng như phân tích ở mục 2.3. Nhóm viết lại kernel chấm điểm để cộng theo từng cặp (tài liệu, token) rồi mới lấy max theo token và cộng theo token query, và kiểm chứng bằng ba cách tính độc lập ở mục 4.3. Vì lệnh đánh giá mặc định của repo đi qua đúng đường này, con số 10,8 mà hai nhóm trong issue #1 thu được sau khi tự sửa lỗi nạp SAE nhiều khả năng cũng bị kéo xuống bởi lỗi này.

Khi đọc checkpoint, nhóm còn thấy hai điểm khác với mô tả trong bài. Chỉ 8.192 trong 16.384 neuron có trọng số khác không, nửa còn lại có trọng số 0 và bias -10, nên checkpoint thực chất có $h = 8.192$. Trên NFCorpus, 7.453 neuron (45,5%) được kích hoạt ít nhất một lần; tác giả nói trong issue rằng hơn 70% neuron được kích hoạt, con số không thể đạt được với checkpoint này.

4.5. Kết quả trên split test và đối chứng train/dev

4.5.1. Split test

Bảng 4.10 so sánh nDCG@10 (có trọng số, chuẩn BEIR) của checkpoint công bố khi chạy bằng mã gốc (chỉ sửa lỗi nạp SAE) và bằng mã đã sửa, cùng với số trong bài và ColBERTv2 đo lại; trích log của một lần chạy ở Phụ lục A.

Bảng 4.10: nDCG@10 trên split test (nhóm đo). "Repo gốc" = checkpoint công bố + mã gốc (đã sửa lỗi nạp SAE); "đã sửa" = MaxSim đúng công thức (4); SSR++ theo cấu hình Phụ lục D.1.

Tập (số tài liệu)	Bài báo	Repo gốc	SSR-tok đã sửa	SSR++ đã sửa	ColBERTv2
NFCorpus (3,6k)	38,8	8,3	43,97	43,98	34,40
SciFact (5,2k)	72,9	7,5	87,83	87,83	69,16
FiQA (57,6k)	39,4	không chạy	41,41	36,86	35,80
Quora (523k)	88,3	không chạy	85,42	không chạy	85,67

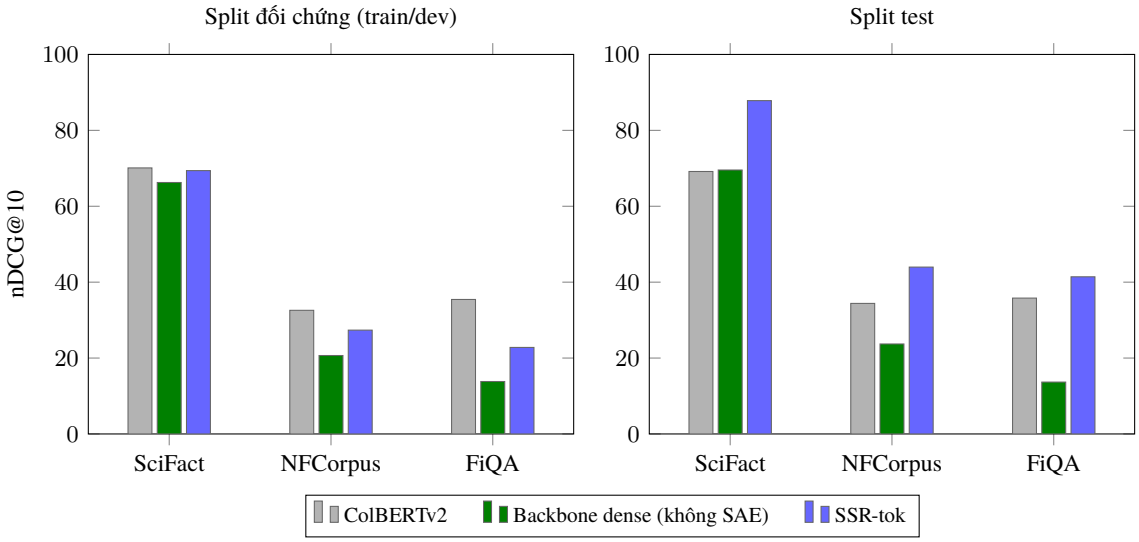
Sau khi sửa, mã chạy đúng công thức và SSR vượt ColBERTv2 trên split test của ba tập đầu, đúng chiều với tuyên bố của bài. Checkpoint công bố cho số *cao hơn* bài báo trên 3/4 tập (SciFact 87,8 so với 72,9), nên checkpoint công bố và mô hình dùng cho Bảng 1 có thể không phải cùng một mô hình; nhóm không xác định được điều này chỉ từ điểm số. SciFact 87,8 với một mô hình cỡ BERT-base cao hơn mọi mô hình 7–8B trong Bảng 2 của chính bài, một dấu hiệu cần kiểm tra thêm (mục 4.5.2). SSR++ không mất điểm trên NFCorpus và SciFact chỉ vì 2.000 ứng viên gần như phủ hết corpus (3,6k đến 5,2k tài liệu); trên FiQA (57,6k) nó mất 4,6 điểm, khác với Bảng 4.4.

4.5.2. Đối chứng bằng split train/dev của cùng tập dữ liệu

Bài báo mô tả đánh giá BEIR là zero-shot: mô hình chỉ học MS MARCO. Để có thêm điểm đối chứng, nhóm đo ba hệ thống trên cùng corpus nhưng với các split train/dev của cùng tập dữ liệu (gọi là *split đối chứng*), bên cạnh split test: ColBERTv2, SSR-tok (có SAE), và "backbone dense" là chính BERT của checkpoint SSR nhưng bỏ qua SAE, chấm MaxSim trên vector 768 chiều.

Bảng 4.11: nDCG@10 trên split đối chứng (train/dev) và split test của cùng một tập (nhóm đo). Quora dùng quy ước BEIR bỏ tài liệu trùng id với query.

Tập (đối chứng → test)	Mô hình	Đối chứng	Test	Test − đối chứng
SciFact (train → test)	ColBERTv2	70,10	69,16	−0,9
	Backbone dense (không SAE)	66,26	69,55	+3,3
	SSR-tok	69,39	87,83	+18,4
NFCorpus (dev → test)	ColBERTv2	32,58	34,40	+1,8
	Backbone dense	20,67	23,71	+3,0
	SSR-tok	27,36	43,97	+16,6
FiQA (dev → test)	ColBERTv2	35,44	35,80	+0,4
	Backbone dense	13,82	13,67	−0,2
	SSR-tok	22,80	41,41	+18,6
Quora (dev → test)	ColBERTv2	85,25	85,67	+0,4
	SSR-tok	82,16	85,42	+3,3



Hình 4.2: Chất lượng trên split đối chứng (trái) và split test (phải) của cùng tập dữ liệu (nhóm đo). Chỉ SSR-tok (có SAE) có bước nhảy lớn ở split test.

Kết quả ở Bảng 4.11 và Hình 4.2 được đọc theo ba mức. *Quan sát đo được:* ColBERTv2 chênh dưới 2 điểm giữa hai loại split; backbone BERT của chính checkpoint SSR cũng chênh dưới 3,5 điểm; SSR-tok cao hơn 16–19 điểm ở split test so với split đối chứng trên SciFact, NFCorpus và FiQA, và cao hơn 3,3 điểm trên Quora. Các split đối chứng khác (NFCorpus train 2.590 query: 28,76; FiQA train 1.000 query: 20,91) cho kết quả tương tự dev. Hệ quả trực tiếp là trên split đối chứng, checkpoint công bố ngang hoặc kém ColBERTv2 (SciFact −0,7; NFCorpus

−5,2; FiQA −12,6; Quora −3,1). *Giả thuyết giải thích:* vì backbone không có hiện tượng này, phần chênh lệch nằm ở tầng SAE; một cách giải thích là SAE từng được huấn luyện hoặc chọn mô hình khi có mặt dữ liệu test của BEIR, một cách khác là phân phối query hoặc nhãn của split test khác split train/dev theo cách có lợi cho sparse code. Nhóm chưa kiểm tra thống kê độ dài query, số tài liệu liên quan, trùng lặp giữa các split, hay khoảng tin cậy của chênh lệch. *Điều chưa kết luận được:* checkpoint công bố có mức cải thiện khác biệt rõ giữa split test và các split train/dev đối chứng trên ba tập dữ liệu đã đo; kiểm tra backbone và baseline chưa giải thích được hiện tượng này; các kết quả hiện tại chưa xác định được nguyên nhân hoặc lịch sử tiếp xúc dữ liệu của checkpoint.

4.6. MS MARCO theo giao thức re-ranking BM25 top-100

MS MARCO là dữ liệu huấn luyện của SSR. Truy hồi toàn bộ 8,84 triệu đoạn văn cần chỉ mục mức token khoảng 70–190 GB RAM, vượt khả năng của máy, nên nhóm dùng giao thức re-ranking: BM25 lấy top-100 cho mỗi query, rồi mọi mô hình chấm lại cùng tập ứng viên bằng MaxSim chính xác. Kết quả bị giới hạn bởi recall của BM25 (66,8% query dev-small có đoạn liên quan trong top-100), nên không so trực tiếp với số full retrieval trong bài; so sánh *giữa các mô hình* trong cùng giao thức là công bằng.

Bảng 4.12: Re-ranking BM25 top-100 trên MS MARCO (nhóm đo). Pipeline SSR được kiểm tra lại: sparse code của query trùng 235/235 dòng với hàm của repo, điểm khớp kernel exact tới $1{,}4 \cdot 10^{-6}$.

Hệ thống	dev-small MRR@10	dev-small nDCG@10	TREC DL19 nDCG@10
BM25 (không re-rank)	18,38	22,83	50,72
ColBERTv2	**35,86**	**40,92**	**72,72**
SSR-tok (checkpoint công bố)	23,18	28,29	57,96
Backbone dense của SSR (không SAE)	15,94	20,34	47,43

Theo Bảng 4.12, trong giao thức này checkpoint công bố kém ColBERTv2 12,7 điểm MRR@10 trên dev-small và 14,8 điểm nDCG@10 trên DL19, cùng chiều với các split đối chứng ở mục 4.5.2. Kết quả này không so được trực tiếp với cột MS của Bảng 4.2 (full retrieval), nhưng cho thấy trong cùng điều kiện, thứ tự giữa hai mô hình ngược với bảng đó. Tầng SAE vẫn có ích so với chính backbone của nó (+7,2 MRR@10), nhưng backbone này là một retriever dense yếu, thấp hơn cả BM25, cho thấy BERT đã bị thay đổi nhiều trong quá trình huấn luyện (tác giả xác nhận không đóng băng BERT).

4.7. So sánh với ColBERTv2/PLAID và giới hạn của phép so sánh

Các tuyên bố về hiệu quả được kiểm tra trên FiQA (57.638 tài liệu, khoảng 9,8 triệu token) với PLAID của PyLate/fast-plaid (nén 2 bit, chạy GPU) theo giao thức ở mục 4.2.2. Bảng 4.13 đưa cả các số của bản tối ưu ở Chương 5 để tiện so sánh; nguồn của từng số ghi ở Phụ lục D.

Bảng 4.13: Hiệu năng trên FiQA (nhóm đo). Thời gian mã hóa corpus không tính vào dựng chỉ mục (PLAID 284 s; SSR khoảng 1.294 s, ngoại suy từ 5.000 tài liệu với batch 16). Độ trễ của SSR trên 648 query test, riêng đường của repo đo trên 100 query đầu.

Hệ thống	Dựng chỉ mục (s)	Chỉ mục (GB)	ms/query TB (p95)	Thiết bị	nDCG test	nDCG dev
ColBERTv2 + PLAID (fast-plaid, 2 bit)	32,6	0,63	44,7 (51)	GPU	35,73	34,93
SSR-tok exact, đường của repo đã sửa	164	2,58	1.635	CPU, 1 luồng	41,41	22,80
SSR-tok exact, `fast_hyb` (nhóm)	2,9–4,8	4,25	54,4 (82)	CPU, 12 luồng	41,41	22,80
SSR-tok LB-rerank t=8, 1.000 ứng viên (nhóm)	như trên	4,25	36,7 (57)	CPU, 12 luồng	41,41	22,81
SSR-tok exact GPU (nhóm)	3,7	3,2 (VRAM)	55,9 (60)	GPU	41,41	22,80
SSR++ cấu hình bài (bản tăng tốc)	như `fast_hyb`	4,25	11,7 (17)	CPU, 12 luồng	36,86	18,11

Bảng 4.14 đối chiếu từng tuyên bố của bài với số đo được. Về chất lượng trên cùng FiQA: trên split dev đối chứng SSR-tok đạt 22,80 so với 34,93 của PLAID; SSR chỉ vượt PLAID trên split test. SSR mã hóa chậm hơn ColBERTv2 vì tài liệu dài tới 512 token (ColBERTv2: 300) và tầng SAE sinh hàng 16.384 chiều cho mỗi token; với batch 32 các tensor này tràn 8 GB VRAM và chậm đi khoảng 8 lần, nên nhóm mã hóa với batch 16.

Bảng 4.14: Đối chiếu từng tuyên bố về hiệu quả của bài báo với số nhóm đo trên FiQA.

Bài báo nói	**Đo được (FiQA, 57.638 tài liệu)**	**Nhận xét**
Dựng chỉ mục nhanh hơn 15 lần vì bỏ K-means (7,3 h so với 122,9 h trên MS MARCO)	K-means + nén của fast-plaid mất 32,6 s trên GPU; dựng posting của SSR mất 2,9–164 s tùy cài đặt; mã hóa corpus (PLAID 284 s, SSR khoảng 1.294 s) lớn hơn nhiều so với dựng chỉ mục	Không quan sát được ở quy mô này với cấu hình đã thử; hai hệ chạy trên thiết bị khác nhau nên chưa tách tác động của thuật toán và phần cứng
PLAID phải dựng lại khi thêm tài liệu, SSR chỉ cần thêm posting	Triển khai fast-plaid mà nhóm dùng có `add_documents`, `remove_documents`, `update_documents`	Triển khai nhóm dùng hỗ trợ cập nhật tài liệu; nhóm chưa đo chất lượng sau cập nhật và không kết luận cho PLAID gốc trong bài
Truy hồi nhanh gần 2 lần PLAID	SSR exact 54–59 ms (CPU 12 luồng) so với PLAID 44,7 ms (GPU); SSR++ 11,7 ms nhưng mất 4,6 điểm nDCG@10	Chỉ SSR++ nhanh hơn rõ, với chi phí chất lượng; so sánh CPU/GPU là so sánh cấu hình thực tế trên máy nhóm
Chỉ mục nhỏ hơn PLAID (18,5 GB so với 22,1 GB)	SSR 2,58–4,25 GB so với PLAID 0,63 GB	Ngược chiều trên FiQA; PLAID nén 2 bit, SSR lưu giá trị float32 và tài liệu dài 512 token
SSR++ hầu như không mất chất lượng (Bảng 5)	FiQA test 41,41 → 36,86; dev 22,80 → 18,11; NFCorpus và SciFact không mất vì 2.000 ứng viên phủ gần hết corpus	Không quan sát được trên FiQA với checkpoint công bố và cấu hình Phụ lục D.1
$h = 16.384$ neuron (Bảng 6)	Checkpoint chỉ có 8.192 neuron có trọng số khác không	Checkpoint khác cấu hình mô tả

4.7.1. Giới hạn của thực nghiệm

- MS MARCO chỉ kiểm bằng re-ranking BM25 top-100; full retrieval cần ≥70 GB RAM cho chỉ mục mức token. Kết luận về MS MARCO chỉ áp dụng trong giao thức này.
- Hiệu năng chỉ đo trên một corpus (FiQA) và một máy; SSR chạy trên CPU còn PLAID chạy trên GPU, nên các so sánh độ trễ và thời gian dựng chỉ mục là so sánh cấu hình thực tế, chưa tách riêng tác động của thuật toán và phần cứng. Độ trễ trên RTX 5060 / i5-12400F không so trực tiếp được với A100 trong bài.
- Chưa huấn luyện lại đầy đủ (khoảng 500k bước GPU), chưa kiểm biến thể Llama-Embed-8B (cần hơn 8 GB VRAM), chưa kiểm SSR-CLS vì không có checkpoint, và chưa triển khai `token_skip_mode`.
- Mọi kết luận về chất lượng áp dụng cho checkpoint công bố trên HuggingFace; nguyên nhân của chênh lệch giữa split test và split đối chứng chưa được xác định.

CHƯƠNG 5. CẢI TIẾN, THẢO LUẬN VÀ KẾT LUẬN

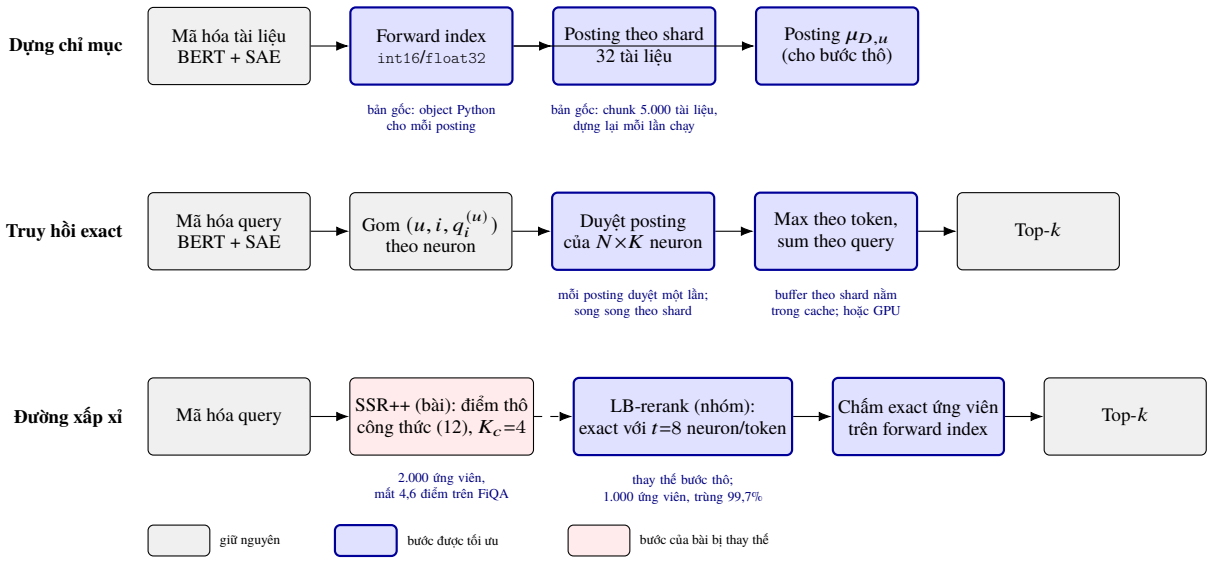
Chương này trình bày phần cải tiến mã nguồn của nhóm (câu hỏi K4): nút thắt đã đo được, kernel exact mới, kết quả trên cùng tập query, phép xấp xỉ LB-rerank và những hướng không hiệu quả. Hai mục cuối thảo luận về phương pháp và trả lời từng câu hỏi kiểm chứng. Toàn bộ mã nằm trong thư mục `opt/` của repo nhóm; giao thức đo theo mục 4.2.2, nguồn từng số liệu ở Phụ lục D.

5.1. Nút thắt đã đo được và mục tiêu tối ưu

Mục tiêu là giảm độ trễ của phép truy hồi exact theo (2.5) mà giữ nguyên kết quả xếp hạng; mọi so sánh chất lượng của các phương án xấp xỉ dùng split đối chứng. Nhóm dùng hai mốc so sánh, cả hai đều là mã của repo sau khi sửa lỗi chấm điểm:

- **Repo exact:** đường exact mặc định của repo, một luồng, corpus chia thành từng chunk 5.000 tài liệu và chỉ mục chunk được dựng lại ở mỗi lần chạy. Trên 100 query đầu của FiQA test, đường này mất 1.635 ms/query cho riêng phần chấm điểm (2.019 ms nếu tính cả dựng chunk), đỉnh RAM 5,24 GB.
- **Base exact:** cùng cấu trúc dữ liệu và kernel của repo nhưng dựng cho toàn corpus trong một chỉ mục, do nhóm dựng để có tham chiếu một luồng không phụ thuộc vào cách chia chunk. Trên cùng 100 query, đường này mất 2.332 ms/query; dựng chỉ mục mất 174 s cho posting token và thêm 107 s cho posting μ, đỉnh RAM 24,45 GB.

Phân tích hai đường này cho thấy ba điểm tốn thời gian: buffer điểm theo (token, tài liệu) có kích thước $N \times 5000$ (hoặc $N \times |\mathcal{D}|$) không nằm trong cache; mỗi neuron của query được duyệt lại một lần cho mỗi token query chứa nó; và chỉ chạy một luồng. Hình 5.1 đánh dấu các bước được tối ưu trong pipeline xử lý một query; Bảng 5.1 liệt kê từng bước với cách làm của bản gốc và của bản tối ưu. Phần mã hóa query (BERT + SAE) và phần chọn top-k giữ nguyên; biểu diễn sparse và công thức chấm điểm cũng giữ nguyên, nên mọi kết quả xếp hạng của đường exact trùng khớp với bản gốc.



Hình 5.1: Pipeline truy hồi của SSR và vị trí các tối ưu của nhóm. Hàng trên: dựng chỉ mục; hàng giữa: truy hồi exact (công thức (4)); hàng dưới: đường xấp xỉ, trong đó bước thô của SSR++ được thay bằng LB-rerank.

Bảng 5.1: Từng bước của pipeline: cách làm của bản gốc (repo đã sửa) và của bản tối ưu (nhóm đo trên FiQA).

Bước	Bản gốc (repo đã sửa)	Bản tối ưu	Kết quả (FiQA)
Dựng chỉ mục	Mỗi posting là một object Python, sắp xếp bằng Python; corpus chia thành từng chunk 5.000 tài liệu và dựng lại ở mỗi lần chạy	Forward index `fw_lat int16[n_tok,K]`, `fw_val float32[n_tok,K]`; posting đếm và điền bằng numba, lưu shard-major (khóa = shard×h + neuron)	2,9–4,8 s thay vì 174 s + 107 s của base exact; RAM đỉnh 7,45 GB thay vì 24,45 GB
Mã hóa query, gom theo neuron	Giữ nguyên	Giữ nguyên	–
Duyệt posting	Với mỗi token query và mỗi neuron của nó, duyệt lại posting của neuron đó (một neuron chung cho m token bị duyệt m lần); một luồng	Mỗi neuron của query duyệt một lần, cập nhật mọi token chứa nó (cập nhật dạng vector khi ≥ 3 token, vô hướng khi ít hơn); các shard chạy song song (`prange`)	Chiếm phần lớn mức tăng tốc 5,0 lần (1 luồng) đến 29,6 lần (12 luồng) so với repo exact
Max theo token, sum theo query	Buffer $N \times 5000$ tài liệu, không nằm trong cache; gộp sau khi duyệt xong cả chunk	Gộp ngay trong shard trên buffer $n_{\text{tok}} \times N$ cỡ vài KB; bản GPU thay hai bước này bằng SpMM (CSR) + `segment_reduce`	GPU: 56 ms/query
Top-k	`argpartition`	Giữ nguyên	–
Sinh ứng viên (đường xấp xỉ)	Điểm thô theo công thức (12) trên posting μ, $K_{\text{coarse}} = 4$, 2.000 ứng viên	LB-rerank: chấm exact với query cắt còn $t = 8$ neuron mỗi token (cận dưới của công thức (4)), 1.000 ứng viên, rồi chấm exact trên forward index	FiQA dev: nDCG@10 22,81 so với 18,11 của SSR++; 0,67 thời gian exact

5.2. Kernel exact `fast_hyb` và các thay đổi cấu trúc dữ liệu

Kernel mới (`opt/ssr_opt/fast_exact.py`) thay đổi bốn điểm:

- **Chia nhỏ theo tài liệu:** corpus chia thành các shard 32 tài liệu liên tiếp; posting được lưu shard-major (khóa = shard ×h + neuron) nên buffer nháp của một shard chỉ vài KB và nằm gọn trong cache L1/L2. Các shard xử lý song song bằng numba `prange`.
- **Duyệt mỗi posting một lần:** trong một shard, với mỗi neuron u của query, duyệt posting (d, t, v) của u đúng một lần và cập nhật *mọi* token query chứa u (bản gốc duyệt lại cho từng token).
- **Forward index:** lưu thêm `fw_lat int16[n_tok, K]` và `fw_val float32[n_tok, K]` để chấm chính xác một tập tài liệu bất kỳ (dùng cho bước tinh của SSR++ và LB-rerank), cùng posting μ cho bước thô.
- **Giữ thứ tự cộng:** các tích được cộng theo thứ tự neuron tăng dần như kernel gốc, nên điểm số giống hệt từng bit; test `opt/tests/test_fast_exact.py` kiểm tra chênh lệch bằng 0 trên mọi tài liệu và mọi query của NFCorpus và SciFact.

Chi phí của cách tổ chức này là bộ nhớ. Chỉ mục FiQA của bản tối ưu chiếm 4,25 GB (forward index 1,57 GB, posting μ 0,47 GB, posting token 2,22 GB), lớn hơn chỉ mục 2,58 GB của đường repo khoảng 1,6 lần vì lưu thêm hai cấu trúc phục vụ bước tinh và bước thô. RAM đỉnh khi dựng và chạy là 7,45 GB, cao hơn đường repo chia chunk (5,24 GB) nhưng thấp hơn nhiều so với base exact (24,45 GB). Thời gian dựng chỉ mục FiQA dao động 2,6–4,8 s giữa các lần chạy, so với 164 s của đường repo.

5.3. Kết quả trên cùng tập query: một luồng, nhiều luồng, GPU

Vì đường repo chỉ chạy được trên 100 query đầu của FiQA test trong thời gian hợp lý, mọi tỉ số tăng tốc trên FiQA được tính trên đúng 100 query đó (Bảng 5.2); phân bố độ trễ trên toàn bộ 648 query test được báo cáo riêng (Bảng 5.3). Trên SciFact và NFCorpus, đường repo chạy được toàn bộ query test nên tỉ số tính trên toàn tập. Hình 5.2 vẽ các số của Bảng 5.2.

Hai điểm cần giải thích trong các bảng trên. Mức tăng tốc một luồng (5,0 lần) đến từ cấu trúc dữ liệu và thứ tự vòng lặp; phần còn lại đến từ song song hóa theo shard: 6 luồng (6 nhân vật lý) nhanh hơn một luồng 3,9 lần, và 12 luồng (siêu phân luồng) chỉ nhanh thêm 1,5 lần, cho thấy kernel bị giới hạn bởi băng thông bộ nhớ hơn là bởi tính toán. Trên NFCorpus test, đường repo cho nDCG@10 43,97 còn base exact và `fast_hyb` cho 43,99: hai đường

Bảng 5.2: Độ trễ của các kernel exact trên cùng tập query (nhóm đo). Mọi phương án exact cho top-100 giống hệt bản gốc (bitwise với `fast_hyb`, trùng khi xét hòa điểm với GPU float32); SSR++ là phép xấp xỉ (nDCG@10 37,58 so với 45,36 trên 100 query này).

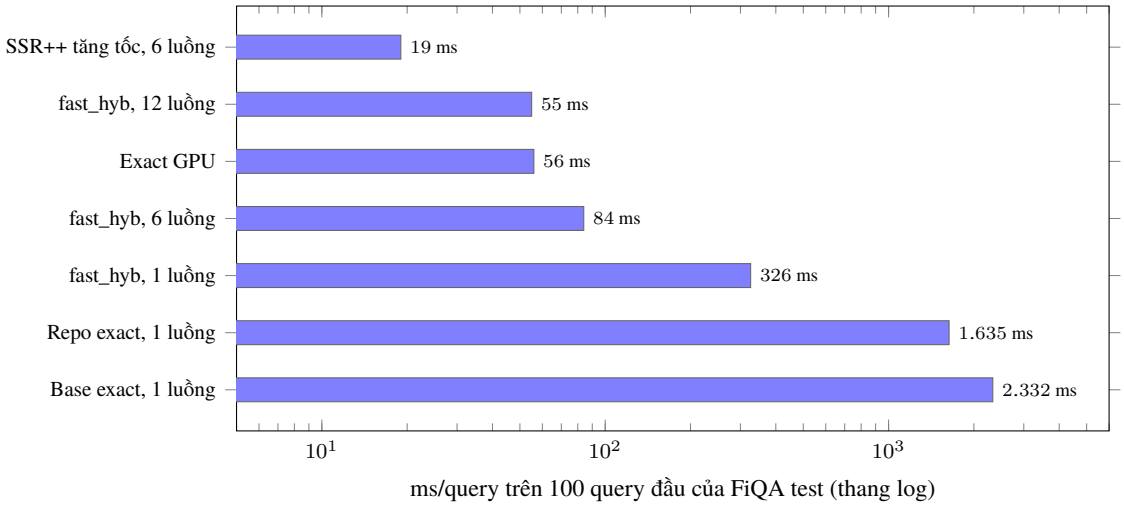
Tập (query)	Phương pháp	Luồng	ms/query	So với repo	So với base
FiQA test (100 đầu)	Repo exact (chunk 5.000)	1	1.635	1,0	1,4
FiQA test (100 đầu)	Base exact (một chỉ mục)	1	2.332	0,7	1,0
FiQA test (100 đầu)	`fast_hyb`	1	326	5,0	7,2
FiQA test (100 đầu)	`fast_hyb`	6	84	19,4	27,7
FiQA test (100 đầu)	`fast_hyb`	12	55	29,6	42,2
FiQA test (100 đầu)	Exact GPU (CSR + segment max)	GPU	56	29,3	41,7
FiQA test (100 đầu)	SSR++ cấu hình bài, bản tăng tốc (xấp xỉ)	6	19	86	123
SciFact test (300)	Repo exact → `fast_hyb`	1 → 12	354 → 12,8	27,6	28,6
SciFact test (300)	Exact GPU	GPU	11,6	30,5	31,6
NFCorpus test (323)	Repo exact → `fast_hyb`	1 → 12	43,6 → 2,5	17,2	17,4
NFCorpus test (323)	Exact GPU	GPU	7,5	5,8	5,9

Bảng 5.3: Phân bố độ trễ trên toàn bộ 648 query FiQA test (nhóm đo, ms/query).

Phương pháp	Luồng	nDCG@10	Trung bình	p50	p95
`fast_hyb`	1	41,41	320,4	318,5	496,6
`fast_hyb`	6	41,41	82,3	81,6	127,2
`fast_hyb`	12	41,41	54,4	53,0	82,1
Exact GPU	GPU	41,41	55,9	55,5	59,6
SSR++ cấu hình bài (bản tăng tốc)	12	36,86	11,7	11,4	16,9

cho cùng tập top-100 khi xét hòa điểm (trùng 100%), nhưng cách chia chunk của repo xếp các tài liệu bằng điểm theo thứ tự khác, nên nDCG chênh 0,02.

Kernel exact trên GPU. Nhóm cài thêm một bản exact trên GPU (`opt/ssr_opt/gpu_exact.py`): chỉ mục lưu dạng CSR (chỉ số int64), tích giữa query và token tính bằng SpMM rồi lấy max theo tài liệu bằng `torch.segment_reduce`. Hai điểm cần chú ý khi chạy trên Windows: phải mã hóa toàn bộ query trước rồi giải phóng mô hình để tránh tràn VRAM sang bộ nhớ chia sẻ (chậm 8 lần), và `segment_reduce` nhanh hơn nhiều so với `scatter_reduce` với tensor mở rộng. Kết quả 56 ms/query trên FiQA, ngang bản CPU 12 luồng, sai số số học cỡ 10^{-6} (float32) và top-100 trùng hoàn toàn khi xét đến hòa điểm; chỉ mục chiếm 3,2 GB VRAM.



Hình 5.2: Độ trễ truy vấn trên cùng 100 query đầu của FiQA test (nhóm đo, số luồng ghi trên nhãn). Mọi phương án exact cho kết quả xếp hạng giống hệt bản gốc; SSR++ là phép xấp xỉ.

5.4. Phương án xấp xỉ LB-rerank

SSR++ chấm điểm thô bằng (3.3), tức bỏ $K - K_{\text{coarse}} = 28$ neuron của mỗi token query và bỏ ràng buộc "cùng một token tài liệu", nên trên FiQA mất nhiều ứng viên (mục 3.3.2). Nhóm đề xuất *LB-rerank*: xếp ứng viên bằng MaxSim *chính xác* của phiên bản query chỉ giữ t neuron lớn nhất của mỗi token, rồi chấm exact (đủ K) trên 1.000 ứng viên bằng forward index. Vì mọi giá trị không âm, điểm ở bước chọn ứng viên là một *cận dưới* của (2.5) và vẫn ràng buộc các neuron thuộc cùng một token tài liệu, nên xếp hạng gần với điểm thật hơn bước thô của SSR++. Cận dưới này không bảo đảm tập ứng viên chứa mọi tài liệu của top-k exact (một tài liệu có điểm cao nhờ các neuron ngoài top-t có thể bị bỏ sót); chất lượng chọn ứng viên được kiểm chứng bằng thực nghiệm ở Bảng 5.4.

Bảng 5.4: Các phương án xấp xỉ trên FiQA dev (nhóm đo, 500 query, 6 luồng; exact = 82,2 ms, nDCG@10 22,80).

Phương án	nDCG@10	Recall@100	Trùng top-10 với exact	Thời gian / exact
SSR++ cấu hình bài (K_{coarse} 4, 2.000 ứng viên)	18,11	33,10	0,59	0,22
SSR++ K_{coarse} 4, 10.000 ứng viên	22,25	48,74	0,88	0,73
SSR++ K_{coarse} 32, 2.000 ứng viên	16,30	29,83	0,51	0,37
LB-rerank t = 8, 1.000 ứng viên	**22,81**	**50,72**	**0,997**	0,67
LB-rerank t = 4, 1.000 ứng viên	22,40	46,23	0,91	0,52

LB-rerank $t = 8$ trùng 99,7% top-10 với exact ở 0,67 lần thời gian; FiQA train (1.000 query) cho cùng xu hướng (exact 20,91; LB-rerank 20,91; SSR++ cấu hình bài 17,90). Tăng K_{coarse} lên 32, tức dùng đủ neuron trong bước thô, còn làm SSR++ kém hơn, cho thấy vấn đề của bước thô nằm ở việc mất ràng buộc "cùng một token tài liệu" chứ không phải thiếu neuron. Trên FiQA test với 12 luồng, LB-rerank đạt 36,7 ms/query và giữ nguyên nDCG@10 41,41 (Bảng 4.13).

5.5. Những hướng đã thử nhưng không hiệu quả

- **Dừng sớm an toàn bằng cận trên** (công thức (3.3) với đủ K neuron, kết hợp chặn Cauchy-Schwarz theo chuẩn token): đúng về toán và cho top-k giống hệt (`fast_ub`, 207 ms trên 100 query FiQA với 6 luồng), nhưng chậm hơn `fast_hyb` cùng số luồng 2,5 lần vì cận quá lỏng. Bảng 5.5 giải thích: ngưỡng để lọt top-100 chỉ khoảng 0,22–0,32 cho mỗi token query, trong khi độ lỏng của cận trên là 0,34–0,51, nên 97–99% tài liệu vẫn nằm trên ngưỡng và không thể bị bỏ qua.
- **Pruning theo block (block-max)** như bài mô tả: cận theo block còn lỏng hơn cận theo tài liệu nên nhóm không triển khai tiếp; với checkpoint này, cơ chế "bỏ qua block theo U_B" khó đóng góp nhiều vào tốc độ.
- **Gộp nhiều query một lần:** tăng thông lượng tối đa khoảng 1,2 lần.

Bảng 5.5: Độ chặt của cận trên dùng để pruning (nhóm đo trên 40 query mỗi tập). θ: ngưỡng top-100 trung bình mỗi token query; "UB $\geq \theta$": tỉ lệ tài liệu có cận trên vượt ngưỡng (không bỏ qua được).

Tập	θ mỗi token	Cận	UB $\geq \theta$	Độ lỏng mỗi token
NFCorpus test	0,22	posting μ (đủ K)	97,6%	0,34
NFCorpus test	0,22	t = 8 neuron	94,7%	0,23
SciFact train	0,32	posting μ (đủ K)	99,3%	0,51
SciFact train	0,32	t = 8 neuron	98,3%	0,37

5.6. Thảo luận

5.6.1. Điểm mạnh của phương pháp

- **Ý tưởng đơn giản, tận dụng cấu trúc dữ liệu sẵn có:** biểu diễn sparse TopK biến mỗi neuron thành một "từ", cho phép dùng inverted index mà vẫn giữ tương tác giữa các token của ColBERT; không cần

K-means, codebook hay giải nén residual.

- **Chỉ mục cập nhật được:** thêm tài liệu chỉ là thêm posting, không phụ thuộc vào codebook đã học.
- **Tiềm năng tốc độ:** với kernel phù hợp, truy hồi exact đạt 54–59 ms/query trên CPU 12 luồng cho 57k tài liệu và dựng chỉ mục trong vài giây; biểu diễn sparse cũng cho phép xấp xỉ có kiểm soát (LB-rerank) gần như không mất chất lượng.
- **Có phân tích lý thuyết:** cận sai lệch giữa sparse MaxSim và MaxSim dense (mục 2.5) nối chất lượng biểu diễn với các đại lượng tối ưu được trong huấn luyện; phương pháp áp dụng cho cả BERT lẫn backbone LLM 8B.

5.6.2. Hạn chế của bài báo và của checkpoint công bố

- **Không tái lập được từ mã nguồn công bố:** mã chấm điểm sai công thức (4), hàm nạp checkpoint tạo SAE ngẫu nhiên, thiếu số hạng loss, data loader khác mô tả của tác giả; tác giả xác nhận cách huấn luyện thật (lr, số negative, nhiệt độ) khác Bảng 6.
- **Chênh lệch chưa giải thích được giữa split test và split đối chứng:** checkpoint công bố cao hơn 16–19 điểm ở split test của ba tập BEIR, trong khi ColBERTv2 và backbone của chính nó không có hiện tượng này; trên split đối chứng và trên MS MARCO re-ranking nó kém ColBERTv2 từ 0,7 đến 12,7 điểm. Nhóm ghi nhận hiện tượng và các giả thuyết (mục 4.5.2) nhưng chưa xác định được nguyên nhân. Checkpoint cũng chỉ dùng 8.192/16.384 neuron, khác cấu hình trong bài.
- **So sánh lệch chỉ số trên MS MARCO** ở bản v3 và bản PMLR; sau khi v4 sửa, lợi thế in-domain còn +1,0.
- **Các tuyên bố hiệu quả chưa quan sát được ở quy mô nhóm đo:** trên FiQA 57.638 tài liệu, thời gian dựng chỉ mục bị chi phối bởi mã hóa, chỉ mục lớn hơn PLAID 4–7 lần, và SSR++ mất 4,6 điểm. Các kết quả này đo trên một corpus với SSR trên CPU và PLAID trên GPU, nên chưa đủ để kết luận cho quy mô MS MARCO trong bài.
- **Những điểm chưa rõ trong bài:** cận lý thuyết chưa được kiểm chứng định lượng qua ϵ và δ; điều kiện bỏ block của SSR++ không được viết rõ; cách kết hợp điểm `[CLS]` với điểm token không được mô tả; K tối ưu thay đổi theo miền; trên miền tranh luận và quan điểm SSR còn kém SPLADE.
- **Chi phí bộ nhớ khi mã hóa:** tầng SAE sinh hàng 16.384 chiều cho mỗi token, dễ tràn VRAM với batch lớn hoặc tài liệu dài.

5.6.3. Điều kiện triển khai

Vì chỉ mục của SSR là inverted index thông thường, phương pháp có thể nhúng vào các máy tìm kiếm sẵn có (Lucene/Elasticsearch) dưới dạng pseudo-token, tương tự cách SPLADE được triển khai, và phù hợp với kho tài liệu thay đổi liên tục hoặc bước truy hồi của hệ thống RAG cần độ trễ vài chục ms trên CPU. Điều kiện đi kèm là một checkpoint được huấn luyện minh bạch và được đánh giá trên split đối chứng; với checkpoint hiện tại, chất lượng trên split đối chứng chưa bằng ColBERTv2, và chỉ mục tốn bộ nhớ hơn PLAID. Như phần tác động của bài báo lưu ý, truy hồi nhanh hơn cũng khuếch đại thiên lệch xếp hạng sẵn có, nên triển khai cần kèm kiểm tra thiên lệch và giám sát kết quả ở các tác vụ tiếp theo.

5.7. Kết luận và hướng phát triển

5.7.1. Kết luận theo từng câu hỏi kiểm chứng

Về cơ chế, báo cáo đã trình bày SSR như một cách tổ chức MVR không cần K-means: chiếu token lên không gian sparse bằng TopK Sparse Autoencoder, dùng mỗi neuron hoạt động làm pseudo-token của inverted index, chấm điểm bằng sparse MaxSim với sai lệch so với điểm dense bị chặn bởi ϵ, δ và C, và huấn luyện SAE với hàm mục tiêu hybrid training trong đó $\mathcal{L}_{\text{CE}}$ đóng góp lớn nhất. Bốn câu hỏi kiểm chứng ở mục 1.7 được trả lời như sau.

(K1) Tái lập. Mã nguồn công bố không tái lập được Bảng 1: với checkpoint công bố, mã gốc cho nDCG@10 8,3 trên NFCorpus và 7,5 trên SciFact. Nhóm phát hiện 10 điểm sai lệch so với bài, đã xử lý 9 điểm bằng bản sửa hoặc bổ sung cấu hình (patch 16 file kèm unit test); sau sửa, checkpoint cho 43,97 và 87,83, cao hơn số trong bài. Cơ chế `token_skip_mode` chưa được triển khai và nhóm chưa huấn luyện lại, nên tái lập đầy đủ từ đầu vẫn chưa thực hiện được.

(K2) Chất lượng so với đối chứng. Trên cùng pipeline, checkpoint công bố vượt ColBERTv2 trên split test của NFCorpus, SciFact và FiQA, nhưng kém ColBERTv2 từ 0,7 đến 12,6 điểm trên các split train/dev đối chứng của cùng tập và kém 12,7 điểm MRR@10 trên MS MARCO theo giao thức re-ranking BM25 top-100. Chênh lệch 16–19 điểm giữa split test và split đối chứng chỉ xuất hiện ở tầng SAE; các phép kiểm tra hiện có chưa xác định được nguyên nhân.

(K3) Hiệu quả. Ở quy mô FiQA 57.638 tài liệu, nhóm không quan sát được lợi thế dựng chỉ mục so với PLAID (mã hóa chiếm phần lớn thời gian, chỉ mục lớn hơn 4–7 lần), và SSR++ với cấu hình của bài mất 4,6 điểm nDCG@10. Các số này đo trên một corpus với SSR trên CPU và PLAID trên GPU, nên là bằng chứng ở quy mô nhóm đo được, không phải phản bác cho quy mô MS MARCO.

(K4) Cải tiến. Kernel `fast_hyb` cho kết quả giống hệt từng bit và nhanh hơn 5,0 lần với một luồng, 29,6 lần với 12 luồng so với exact của repo trên cùng 100 query FiQA; bản GPU tương đương bản CPU 12 luồng; chỉ mục dựng trong vài giây thay vì vài phút, đổi lại chỉ mục lớn hơn khoảng 1,6 lần. LB-rerank thay bước thô của SSR++ giữ 99,7% top-10 của exact với 0,67 lần thời gian trên FiQA dev. Pruning an toàn bằng cận trên không hiệu quả với checkpoint này vì cận quá lỏng.

Các giới hạn còn ảnh hưởng đến kết luận: `token_skip_mode` chưa triển khai, chưa huấn luyện lại đầy đủ, MS MARCO chỉ kiểm bằng re-ranking, chưa kiểm SSR-CLS và backbone LLM, và hiệu năng chỉ đo trên một corpus. Qua đề tài, nhóm rút ra rằng kết quả công bố cần được kiểm chứng bằng baseline đối chứng trên cùng pipeline, bằng split đối chứng bên cạnh split test, và bằng cách đối chiếu mã nguồn với từng công thức trong bài.

5.7.2. Hướng phát triển

Ba hướng ưu tiên, theo thứ tự:

1. **Checkpoint minh bạch:** huấn luyện lại bằng mã đã sửa (đủ số hạng $4k$, 32 negative gom theo query, $\tau = 0{,}3$) trên GPU lớn hơn, công bố log huấn luyện, và đo ϵ, δ của SAE để kiểm chứng định lượng cận lý thuyết.

2. **Đánh giá cùng giao thức ở quy mô lớn hơn:** full retrieval trên MS MARCO, so với PLAID trên cùng thiết bị, đánh giá trên split đối chứng bên cạnh split test, kèm thống kê độ dài query và nhãn để giải thích chênh lệch theo split.

3. **Giảm bộ nhớ chỉ mục:** quantization giá trị posting (8 bit), nén chỉ số token, hoặc giảm độ dài tài liệu về 300 token như ColBERTv2, để thu hẹp khoảng cách với PLAID mà vẫn giữ kernel exact.

Các hướng khác có thể xét sau: sparsity thích ứng theo phân bố kích hoạt của từng token, kết hợp SSR-CLS hoặc BM25 cho miền tranh luận, và thử SAE trên các mô hình embedding đa ngôn ngữ cho tiếng Việt.

TÀI LIỆU THAM KHẢO

[1] L. Guo, Y. Wang, T. Wen, A. Feng, S. Jegelka, C. You, "No More K-means: Single-Stage Sparse Coding for Efficient Multi-Vector Retrieval," *Proc. 43rd International Conference on Machine Learning (ICML)*, PMLR 306, 2026. arXiv:2605.30120 (v3 03/06/2026; v4 28/09/2026).

[2] O. Khattab, M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," *Proc. SIGIR*, 2020, pp. 39–48.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," *Proc. NAACL*, 2022, pp. 3715–3734.

[4] K. Santhanam, O. Khattab, C. Potts, M. Zaharia, "PLAID: An Efficient Engine for Late Interaction Retrieval," *Proc. CIKM*, 2022, pp. 1747–1756.

[5] L. Gao, Z. Dai, J. Callan, "COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List," *Proc. NAACL*, 2021.

[6] J. Lee, Z. Dai, S. M. K. Duddu, T. Lei, I. Naim, M.-W. Chang, V. Zhao, "Rethinking the Role of Token Retrieval in Multi-Vector Retrieval," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[7] J. L. Scheerer, M. Zaharia, C. Potts, G. Alonso, O. Khattab, "WARP: An Efficient Engine for Multi-Vector Retrieval," *Proc. SIGIR*, 2025, pp. 2504–2512.

[8] F. M. Nardini, C. Rulli, R. Venturini, "Efficient Multi-Vector Dense Retrieval with Bit Vectors," *Proc. ECIR*, 2024.

[9] T. Formal, B. Piwowarski, S. Clinchant, "SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking," *Proc. SIGIR*, 2021; T. Formal, C. Lassance, B. Piwowarski, S. Clinchant, "SPLADE v2," arXiv:2109.10086, 2021; C. Lassance, H. Déjean, T. Formal, S. Clinchant, "SPLADE-v3: New Baselines for SPLADE," arXiv:2403.06789, 2024.

[10] L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, J. Wu, "Scaling and Evaluating Sparse Autoencoders," arXiv:2406.04093, 2024 (ICLR 2025).

[11] H. Lee, A. Battle, R. Raina, A. Y. Ng, "Efficient Sparse Coding Algorithms," *Advances in Neural Information Processing Systems*, vol. 19, 2006.

[12] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," *NeurIPS Datasets and Benchmarks*, 2021.

[13] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," *CoCo@NeurIPS*, 2016.

[14] O. Weller, M. Boratko, I. Naim, J. Lee, "On the Theoretical Limitations of Embedding-Based Retrieval," arXiv:2508.21038, 2025.

[15] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, M. Gatford, "Okapi at TREC-3," *Proc. TREC-3*, 1995.

[16] K. Järvelin, J. Kekäläinen, "Cumulated Gain-based Evaluation of IR Techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[17] P. Yang, H. Fang, J. Lin, "Anserini: Enabling the Use of Lucene for Information Retrieval Research," *Proc. SIGIR*, 2017, pp. 1253–1256.

[18] A. Chaffin, R. Sourty, "PyLate: Flexible Training and Retrieval for Late Interaction Models," LightOn, 2024. https://github.com/lightonai/pylate.

[19] Mã nguồn chính thức của SSR: https://github.com/Y-Research-SBU/SSR, commit `c64bc05` (26/06/2026), truy cập 12/10/2026. Issue #1 "Difficulty reproducing BEIR numbers from Table 1" (24/07/2026), https://github.com/Y-Research-SBU/SSR/issues/1; Issue #2 "Is the 4k reconstruction loss missing from the training objective?" (04/08/2026), https://github.com/Y-Research-SBU/SSR/issues/2.

[20] Checkpoint SSR-tok: https://huggingface.co/Y-Research-Group/SSR, đưa lên 05/10/2026, nhóm tải ngày 11/10/2026.

[21] Bản chính thức trên PMLR: https://proceedings.mlr.press/v306/guo26x.html, truy cập 12/10/2026.

[22] Mã nguồn, patch, script đánh giá, kết quả và log của nhóm: https://github.com/duykhanhxx03/ssr-repro-and-optimization.

PHỤ LỤC A. CHI TIẾT CÁC SAI LỆCH TRONG MÃ NGUỒN VÀ BẢN SỬA

Phụ lục này trình bày chi tiết các điểm đã tóm tắt trong Bảng 4.9 (mục 4.4), kèm đoạn mã trước và sau khi sửa, và trích log của các lần chạy được dẫn trong thân bài. Toàn bộ bản sửa nằm trong patch `ssr_paper_fixes.patch` của repo nhóm, áp dụng được lên commit `c64bc05` của mã nguồn gốc bằng `git apply`.

A.1. Điểm 2: nạp checkpoint tạo SAE ngẫu nhiên

Hàm `load_ssr` gọi `SSR(model_name_or_path=...)`; vì PyLate chỉ giữ các module `Transformer`/`Dense` khi nạp, module `SparseAutoencoder` bị bỏ và một SAE mới với $4 \times 768 = 3072$ latent được khởi tạo ngẫu nhiên. Mọi đánh giá bằng lệnh mặc định của repo do đó chạy trên một SAE chưa huấn luyện. Một người dùng trong issue #1 cũng phát hiện lỗi này độc lập.

Listing A.1: `ssr/retrieval/eval_mteb.py`: trước (trên) và sau khi sửa (dưới).

```python
def load_ssr(model_path: Path, device: str) -> SSR:
    model = SSR(model_name_or_path=str(model_path), device=device)   # SAE ngau nhien
    model.eval()
    return model

def load_ssr(model_path: Path, device: str, token_scope=None) -> SSR:
    # SSR(model_name_or_path=...) would attach a randomly initialised SAE.
    return load_ssr_checkpoint(str(model_path), device=device,
                               token_scope=token_scope)
```

Hàm mới `load_ssr_checkpoint` trong `ssr/model.py` đọc `modules.json`, nạp SAE từ thư mục con `1_SparseAutoencoder` (lọc các khóa config lạ), và khôi phục các thiết lập ColBERT lưu trong `config_sentence_transformers.json`: tiền tố `[Q]`/`[D]`, độ dài query 192 và tài liệu 512, `do_query_expansion = false`, danh sách skiplist.

A.2. Điểm 1: công thức chấm điểm trên inverted index

Điểm này ảnh hưởng lớn nhất đến kết quả và chưa được nêu trong các issue. Bộ cộng dồn của chế độ `exact` làm việc theo từng neuron: với mỗi neuron u của token query i, nó lấy giá trị lớn nhất của u trong mỗi tài liệu rồi cập nhật ô (i, D) bằng phép `maximum`:

Listing A.2: `inverted_index.py` gốc: bộ cộng dồn lấy max theo từng neuron.

```python
for q_entries, s, e in segments:          # moi segment = mot neuron u
    docs = doc_idx[s:e]; vals = values[s:e]
    out_docs, max_vals = segment_max_by_doc(docs, vals)   # mu_{D,u}
    for qt, v_q in q_entries:             # cac token query chua u
        scaled = max_vals * np.float32(v_q)
        row = local[qt]
        row[out_docs] = np.maximum(row[out_docs], scaled)  # max, khong phai cong
```

Kết quả là điểm của cặp (token query i, tài liệu D) bằng $\max_u q_i^{(u)} \mu_{D,u}$: mỗi neuron được phép chọn một token tài liệu khác nhau, và chỉ neuron lớn nhất được giữ. So với (2.5), phép tính này vừa đánh mất phép cộng trên các neuron chung, vừa không ràng buộc các neuron phải thuộc cùng một token tài liệu. Kết quả đo được: NFCorpus 8,33 và SciFact 7,46 nDCG@10, thấp hơn cả BM25.

Bản sửa (`ssr/retrieval/exact_index_maxsim.py`, kernel numba) cộng dồn theo đúng hàng (tài liệu, token tài liệu), rồi mới lấy max theo token trong tài liệu và cộng theo token query:

Listing A.3: Kernel exact mới (rút gọn): cộng theo (doc, token) rồi mới max theo token.

```
for qt in range(n_qt):                      # moi token query
    gen_state[0] += 1; gen = gen_state[0]; n_rows = 0
    for k in range(qt_offsets[qt], qt_offsets[qt + 1]):   # moi neuron u cua qt
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(offsets[li], offsets[li + 1]):     # posting (doc, token, val)
            d = doc_idx[p]
            r = row_start[d] + token_idx[p]               # hang = (doc, token)
            if row_gen[r] != gen:                         # lan dau cham hang nay
                row_gen[r] = gen; acc[r] = 0.0
                touched_rows[n_rows] = r; n_rows += 1
            acc[r] += vq * values[p]                      # SUM tren neuron chung
    for i in range(n_rows):                               # MAX theo token trong doc
        r = touched_rows[i]; d = row_doc[r]
        if doc_gen[d] != gen:
            doc_gen[d] = gen; doc_best[d] = acc[r]
            touched_docs[n_docs] = d; n_docs += 1
        elif acc[r] > doc_best[d]:
            doc_best[d] = acc[r]
    for i in range(n_docs):                               # SUM theo token query
        d = touched_docs[i]; doc_total[d] += doc_best[d]
```

Vì mọi giá trị sau TopK+ReLU đều không âm, các token tài liệu không chung neuron nào với token query đóng góp 0 và có thể bỏ qua mà không đổi kết quả max. Bản sửa được kiểm chứng bằng ba cách độc lập: (i) so với MaxSim tính trên vector sparse dạng dense bằng PyTorch (`reference_maxsim_dense`); (ii) so với một cài đặt bằng `scipy.sparse` viết riêng (`verify_scipy.py`); (iii) so với bản "unblocked" của repo đã sửa cùng lỗi (`crosscheck.py`). Cả ba trùng nhau tới 10^{-6} trên mọi cặp query và tài liệu của NFCorpus và SciFact.

A.3. Điểm 3 và 4: ánh xạ neuron và bước thô SSR++

Chỉ mục nén các neuron thực sự xuất hiện trong corpus thành chỉ số liên tiếp (`latent_remap`); đường `e2e-index` quên áp ánh xạ này lên query nên phần giao giữa neuron query và neuron tài liệu chỉ còn khoảng 2% (nDCG@10 1,2 trên NFCorpus). Sau khi ánh xạ query bằng `remap_sparse_embeddings`, kết quả là 44,0, trùng với đường chuẩn.

Bước thô của SSR++ trong repo dùng một cơ chế pruning theo block khác với (3.3) và tham số mặc định ($K_{\text{coarse}} = 8$, 32.768 ứng viên, block 512) không khớp Phụ lục D.1. Nhóm cài đặt lại đúng (3.3) trên posting $\mu_{D,u}$ (được dựng từ posting token bằng một phép sắp xếp), lấy top-2.000 toàn cục trên mọi block, rồi chấm tinh bằng kernel exact với mask `allowed`:

Listing A.4: Kernel chấm điểm thô theo công thức (12).

```
@njit(cache=True)
def _coarse_totals_kernel(mu_offsets, mu_doc, mu_val, seg_rows, seg_vq, doc_total):
    for k in range(seg_rows.shape[0]):        # (neuron u, q_i^{(u)}) voi u trong top-Kcoarse
        li = seg_rows[k]; vq = seg_vq[k]
        for p in range(mu_offsets[li], mu_offsets[li + 1]):
            doc_total[mu_doc[p]] += vq * mu_val[p]     # q_i^{(u)} * mu_{D,u}
```

A.4. Điểm 5–7 và 9: độ đo và huấn luyện

Hàm nDCG của repo đọc qrels thành tập nhị phân, làm mất mức liên quan 2 của NFCorpus và 2–3 của TREC DL; nhóm sửa để giữ `Dict[str, Dict[str, int]]` và dùng độ lợi $2^{\text{rel}} - 1$. Trong `losses.py`, `recons_4k` được tính trong forward của SAE nhưng không đưa vào loss; nhóm thêm thành phần `reconstruction_4k` với hệ số 0,125 (issue #2 xác nhận tác giả có dùng số hạng này). Nhóm cũng thêm `--warmup-steps`, `--negatives-per-query` và một `QueryGroupedBatchSampler` gom các triplet của cùng query vào một batch như tác giả mô tả:

Listing A.5: `ssr/samplers.py`: gom triplet cùng query vào một batch.

```python
class QueryGroupedBatchSampler(DefaultBatchSampler):
    def __init__(self, dataset, batch_size, drop_last, ...):
        super().__init__(dataset, batch_size=batch_size, drop_last=drop_last, ...)
        queries = dataset["query"]; groups = []
        for i, q in enumerate(queries):      # cac hang lien tiep cung query = 1 nhom
            if i > 0 and q == queries[i - 1]: groups[-1].append(i)
            else: groups.append([i])
        self.groups = groups

    def _batches(self):
        g = torch.Generator(); g.manual_seed(int(self.seed) + int(self.epoch))
        order = torch.randperm(len(self.groups), generator=g).tolist()
        batches, current = [], []
        for gi in order:                     # xep nguyen nhom vao batch
            group = self.groups[gi]
            if current and len(current) + len(group) > self.batch_size:
                batches.append(current); current = []
            current = current + group
        if current: batches.append(current)
        return batches
```

Hai preset `scripts/train_table6.sh` (đúng Bảng 6) và `scripts/train_issue1_setup.sh` (cấu hình tác giả nói trong issue: lr 10^{-6}, 32 negative gom theo query, $\tau = 0{,}3$, warmup 20.000) được thêm vào. Đường huấn luyện đã chạy thử trọn vẹn trên một tập MS MARCO nhỏ (batch 16, 8 negative mỗi query; log `logs/train_smoke.log` cho thấy `reconstruction_4k` và warmup lr hoạt động, checkpoint nạp lại được).

A.5. Điểm 8: dựng chỉ mục với corpus lớn

Hàm dựng chỉ mục gốc tạo một object Python cho mỗi posting rồi sắp xếp bằng Python, nên với FiQA (258 triệu posting) tràn RAM 32 GB. Nhóm thay bằng hai lượt đếm rồi điền trên mảng numpy (và kernel numba trong bản tối ưu), đồng thời lọc các khóa lạ trong `config.json` của SAE để checkpoint nạp được. Sau sửa, chỉ mục FiQA dựng trong 164 s bằng đường của repo và 2,9–4,8 s bằng đường tối ưu (Bảng 4.13).

A.6. Điểm 10: `token_skip_mode`

Config của checkpoint có khóa `token_skip_mode` gợi ý một cơ chế bỏ token (stopword) khi mã hóa, nhưng mã nguồn công bố không có hàm nào đọc khóa này. Nhóm giữ cách mã hóa mặc định của mã nguồn (chỉ bỏ các token trong skiplist dấu câu của ColBERT) và ghi nhận đây là điểm chưa xử lý: nếu tác giả bỏ thêm token khi tạo số liệu trong bài, kết quả của nhóm có thể lệch so với cách chạy của tác giả.

A.7. Trích log của các lần chạy được dẫn trong thân bài

Listing A.6: NFCorpus bằng mã đã sửa (`logs/A_fixed_nfcorpus.log`), dẫn ở Bảng 4.10.

```
2026-10-11 20:39:34,255 [INFO] Metrics for nfcorpus: mrr@[] ndcg@[10]
2026-10-11 20:40:02,045 [INFO]   ndcg@10: 0.4397
```

Listing A.7: Đo PLAID trên FiQA (`logs/plaid_bench.log`), dẫn ở Bảng 4.13.

```
[plaid] index cuda: build 32.6s, 603 MiB, encode 284s
[plaid] test cuda: {'ndcg@10': 35.727, 'mean_ms': 44.70, 'p50_ms': 44.44, 'p95_ms': 51.15, 'n_queries': 648}
[plaid] dev cuda:  {'ndcg@10': 34.934, 'mean_ms': 44.18, 'p50_ms': 44.07, 'p95_ms': 50.74, 'n_queries': 648}
```

Listing A.8: Kernel `fast_hyb` trên 648 query FiQA test (`opt/logs/fiqa_test_fast.log`), dẫn ở Bảng 5.3.

```
[fast_hyb t=1]  nDCG@10=41.41 R@100=70.85 mean=320.36ms p50=318.49 p95=496.63 build=4.79 rss=5.14GB
[fast_hyb t=6]  nDCG@10=41.41 R@100=70.85 mean=82.33ms  p50=81.58  p95=127.22 build=4.79 rss=5.21GB
[fast_hyb t=12] nDCG@10=41.41 R@100=70.85 mean=54.40ms  p50=53.01  p95=82.08  build=4.79 rss=5.29GB
```

PHỤ LỤC B. CHỨNG MINH ĐỊNH LÝ A VÀ ĐỊNH LÝ B

Ký hiệu và giả thiết như mục 2.5.

Chứng minh Định lý A. Chứng minh gồm ba bước.

Bước 1: bất đẳng thức tam giác. Tách sai lệch thành phần do reconstruction và phần do cấu trúc decoder:

$$\left|\mathrm{x}^\top \mathrm{y} - \mathrm{z}_x^\top \mathrm{z}_y\right| \le \left|\mathrm{x}^\top \mathrm{y} - \hat{\mathrm{x}}^\top \hat{\mathrm{y}}\right| + \left|\hat{\mathrm{x}}^\top \hat{\mathrm{y}} - \mathrm{z}_x^\top \mathrm{z}_y\right|. \tag{B.1}$$

Bước 2: sai số reconstruction. Đặt $\mathrm{e}_x = \mathrm{x} - \hat{\mathrm{x}}$ và $\mathrm{e}_y = \mathrm{y} - \hat{\mathrm{y}}$. Khai triển $\mathrm{x}^\top \mathrm{y} - \hat{\mathrm{x}}^\top \hat{\mathrm{y}} = \mathrm{x}^\top \mathrm{e}_y + \mathrm{e}_x^\top \mathrm{y} - \mathrm{e}_x^\top \mathrm{e}_y$. Theo bất đẳng thức Cauchy-Schwarz cùng Giả thiết 1 và 2, ba số hạng lần lượt bị chặn bởi $B\epsilon$, ϵB và ϵ^2, nên tổng không vượt quá $2B\epsilon + \epsilon^2$.

Bước 3: sai số hình học của decoder. Vì $\hat{\mathrm{x}} = \mathrm{W}_{\mathrm{dec}} \mathrm{z}_x$ và $\hat{\mathrm{y}} = \mathrm{W}_{\mathrm{dec}} \mathrm{z}_y$ nên $\hat{\mathrm{x}}^\top \hat{\mathrm{y}} = \mathrm{z}_x^\top (\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}}) \mathrm{z}_y$. Trừ đi $\mathrm{z}_x^\top \mathrm{z}_y$ ta được $\mathrm{z}_x^\top (\mathrm{W}_{\mathrm{dec}}^\top \mathrm{W}_{\mathrm{dec}} - \mathrm{I}) \mathrm{z}_y$; theo Giả thiết 3, biểu thức này bị chặn bởi $\delta \|\mathrm{z}_x\|_2 \|\mathrm{z}_y\|_2$, và theo Giả thiết 4 là δC^2. Ghép ba bước lại thu được (2.8). □

Chứng minh Định lý B. Đặt $s_{ij} = \mathrm{q}_i^\top \mathrm{d}_j$ và $\tilde{s}_{ij} = \mathrm{z}_{q_i}^\top \mathrm{z}_{d_j}$. Theo Định lý A, $|s_{ij} - \tilde{s}_{ij}| \le \eta$ với mọi i, j. Dùng bất đẳng thức sơ cấp $|\max_j a_j - \max_j b_j| \le \max_j |a_j - b_j|$, với mỗi token truy vấn sai lệch giữa hai cực đại không vượt quá η. Cộng dồn trên N token truy vấn cho ra (2.9). □

Điều kiện bảo toàn thứ tự. Gọi S và $\tilde{S}$ là điểm dense và điểm sparse của hai tài liệu D_1, D_2 với cùng truy vấn. Nếu $S(D_1) - S(D_2) > 2N\eta$ thì theo (2.9), $\tilde{S}(D_1) \ge S(D_1) - N\eta > S(D_2) + N\eta \ge \tilde{S}(D_2)$, tức thứ tự được giữ. Điều kiện này là đủ, không phải cần.

PHỤ LỤC C. CÁC KẾT QUẢ KHÁC CỦA BÀI BÁO

Phụ lục này tổng hợp các kết quả công bố không trực tiếp phục vụ thực nghiệm của nhóm nhưng được dẫn trong thân bài. Mọi số liệu trong phụ lục này là "theo bài báo" (bản arXiv v3, trừ khi ghi khác).

C.1. Bộ dữ liệu

Bảng C.1: Các bộ dữ liệu dùng trong đánh giá của bài báo.

Bộ	Viết tắt	Lĩnh vực / nhiệm vụ	Vai trò
MS MARCO passage	MS	Truy vấn Bing, 8,84 triệu đoạn	Huấn luyện + in-domain
ArguAna	AR	Phản biện lập luận	BEIR (zero-shot)
Climate-FEVER	CF	Kiểm chứng khí hậu	BEIR
TREC-COVID	CV	Y sinh COVID	BEIR
DBpedia	DB	Tìm thực thể	BEIR
FEVER	FE	Kiểm chứng sự kiện	BEIR
FiQA-2018	FQ	Tài chính	BEIR
HotpotQA	HQ	Hỏi đáp nhiều bước	BEIR
NFCorpus	NF	Y học dinh dưỡng	BEIR
NQ	NQ	Hỏi đáp Google	BEIR
Quora	QU	Câu hỏi trùng lặp	BEIR
SCIDOCS	SD	Trích dẫn khoa học	BEIR
SciFact	SF	Kiểm chứng khoa học	BEIR
Touché-2020	TO	Tranh luận	BEIR
LoTTE	–	Chủ đề đuôi dài, 5 miền	Độ bền long-tail
MS MARCO document	–	Văn bản dài (≈1.131 token)	Văn bản dài
LIMIT	–	Kiểm tra giới hạn biểu diễn	Stress test

Bảng C.1 liệt kê các bộ dữ liệu. BEIR [12] gồm 18 tập dữ liệu truy hồi zero-shot; bài dùng 13 tập. MS MARCO passage [13] có tập dev-small 6.980 truy vấn (nhãn nhị phân) và TREC DL 2019 với 43 truy vấn có nhãn 4 mức. Ngoài MRR@10 và nDCG@10, bài còn dùng Recall@k và Success@5 (cho LoTTE: tỉ lệ truy vấn có ít nhất một tài liệu đúng trong top-5). Độ trễ đo trên GPU A100 40 GB sau 1.000 truy vấn khởi động; các số liệu CPU đo trên Xeon Platinum 8275CL 96 lõi.

C.2. Bảng 1 đầy đủ

C.3. Backbone LLM, LoTTE, văn bản dài và LIMIT

Với backbone Llama-embed-nemotron-8b đóng băng (Bảng C.3), lớp chiếu tuyến tính chỉ cải thiện khoảng 0,3 điểm trong khi sparse coding cải thiện khoảng 1,3 điểm; bài kết luận lợi ích đến từ bản chất sparse coding chứ không phải từ việc thêm tham số. Trên LoTTE (Bảng C.4), SSR vượt ColBERTv2 2,5 và 2,2 điểm với truy vấn Search và Forum. Với văn bản dài trên MS MARCO Document, SSR-CLS đạt MRR@10 48,8 với độ trễ 29,3 ms và SSR-tok đạt 48,3 với 27,5 ms, trong khi ColBERTv2 đạt 41,5 với 79,3 ms. Trên LIMIT (Bảng C.5), các mô hình SVR mạnh nhất chỉ đạt Recall@5 dưới 5%, trong khi SSR đạt 78,6.

C.4. Hiệu quả trên CPU và theo quy mô dữ liệu

Hình C.1 tái hiện Hình 3 của bài: thời gian toàn pipeline và ảnh hưởng của quy mô kho dữ liệu (khi mở rộng từ 0,5 triệu lên 10 triệu tài liệu, SSR-tok giảm 5,3% chất lượng so với 8,9% của ColBERTv2 theo bản v3; bản v4 sửa thành 4,2% so với 4,7%).

Bảng C.2: Bảng 1 của bài báo (bản arXiv v3): nDCG@10 trên 13 tập BEIR, cột MS là MS MARCO (xem mục 4.1.4 về độ đo của cột này), độ trễ ms/truy vấn trên MS MARCO. In đậm: tốt nhất; gạch chân: thứ hai.

Phương pháp	Độ trễ	MS	AR	CF	CV	DB	FE	FQ	HQ	NF	NQ	QU	SD	SF	TO	TB.
Dense retrieval với late interaction																
ColBERT	57,3	36,0	23,6	18,1	67,9	39,2	76,9	31,7	59,2	30,7	51,8	85,7	14,6	66,8	20,5	41,5
COIL	12,6	35,3	29,5	21,7	66,8	39,9	**83,8**	31,3	**71,3**	33,2	52,1	83,8	15,5	70,7	28,1	47,4
ColBERTv2	56,9	39,8	46,3	17,3	73,5	44,9	78,7	36,1	66,6	33,5	56,3	85,3	15,2	69,1	26,1	49,2
PLAID	37,1	39,7	46,1	17,4	73,7	44,7	78,8	36,3	66,7	33,8	56,4	85,0	15,4	69,2	26,3	49,3
AligneR	51,8	38,8	28,9	18,2	68,8	41,6	72,4	33,5	61,5	34,0	52,4	82,3	14,3	70,3	34,8	46,6
CITADEL	15,7	39,9	**51,1**	18,3	71,5	42,2	76,6	33,0	66,3	33,7	54,0	85,3	15,9	69,0	34,2	49,4
XTR	33,4	45,0	40,7	20,8	73,6	40,8	73,7	34,8	64,7	34,0	52,8	85,9	14,6	71,1	31,3	48,8
Learned sparse retrieval																
SPLADE-v2	16,3	43,3	47,6	**23,5**	71,5	43,4	78,7	33,8	68,4	33,5	52,1	83,8	15,9	69,4	**36,3**	50,1
SPLADE-v3	16,6	43,9	50,8	23,3	74,8	45,2	79,8	37,5	69,2	35,8	58,6	81,4	15,8	71,0	29,6	51,2
Sparse retrieval với late interaction																
SSR-tok	17,5	45,2	46,1	22,8	76,4	48,2	81,9	39,4	69,5	38,8	60,7	88,3	17,5	72,9	33,1	52,9
SSR-CLS	19,5	**45,5**	46,6	**23,5**	**76,8**	**48,4**	82,8	**40,0**	69,9	**39,1**	**61,2**	**88,7**	**17,9**	**73,3**	33,7	**53,4**

Bảng C.3: Kết quả theo bài báo khi dùng Llama-embed-nemotron-8b làm backbone đóng băng (trung bình nDCG@10 trên 14 tập).

Mô hình	Trung bình
Llama-embed-nemotron-8b (gốc)	65,8
Qwen3-Embedding-8B	66,0
e5-mistral-7b	58,8
Llama-8B + lớp chiếu tuyến tính	66,1
Llama-8B + SSR-tok	66,4
Llama-8B + SSR-CLS	**67,1**

Bảng C.4: Success@5 trên LoTTE theo bài báo cho hai loại truy vấn Search và Forum.

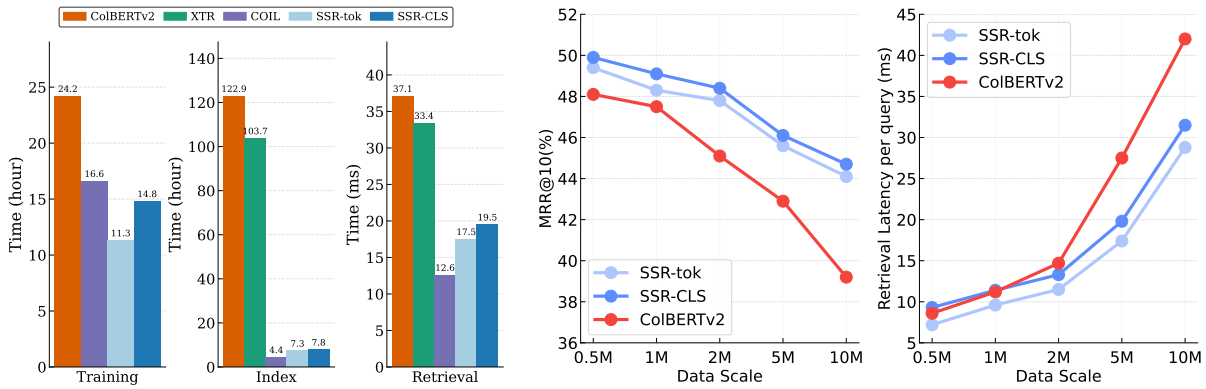
Loại	Mô hình	Writing	Recreation	Science	Technology	Lifestyle	Pooled	TB.
Search	ColBERTv2	79,2	70,8	55,5	65,9	84,3	71,8	71,3
	XTR	78,6	69,5	56,1	64,6	83,5	69,3	70,3
	SSR	**81,8**	**73,9**	**59,3**	**67,1**	**87,6**	**73,3**	**73,8**
Forum	ColBERTv2	78,7	71,4	45,8	53,9	77,3	63,5	65,1
	XTR	76,4	70,9	44,2	53,4	75,8	61,7	63,6
	SSR	**81,8**	**74,3**	**47,3**	**55,9**	**79,7**	**67,4**	**67,3**

Bảng C.5: Kết quả trên bộ LIMIT theo bài báo.

Phương pháp	Recall@5	Recall@10	Recall@100
Qwen3-4B (SVR)	1,2	4,6	6,3
GritLM-7B (SVR)	4,7	6,1	19,4
e5-mistral-7B (SVR)	2,8	5,2	10,9
ColBERTv2 (MVR)	71,8	82,6	94,4
XTR (MVR)	68,4	79,9	93,2
SSR	**78,6**	**87,3**	**98,1**

Bảng C.6: So sánh hiệu quả trên CPU theo bài báo (MS MARCO passage; chỉ mục tính bằng giờ, truy hồi tính bằng ms/truy vấn).

Phương pháp	MRR@10	Lập chỉ mục (h)	Truy hồi (ms)
PLAID	39,6	122,9	156,3
WARP	38,7	103,7	129,7
EMVB	39,8	61,5	93,4
DESSERT	37,2	49,8	75,6
MUVERA	39,7	82,6	94,7
IGP	39,1	119,7	113,2
SSR-tok	39,5	**7,3**	**49,1**
SSR-CLS	**40,2**	7,8	57,3

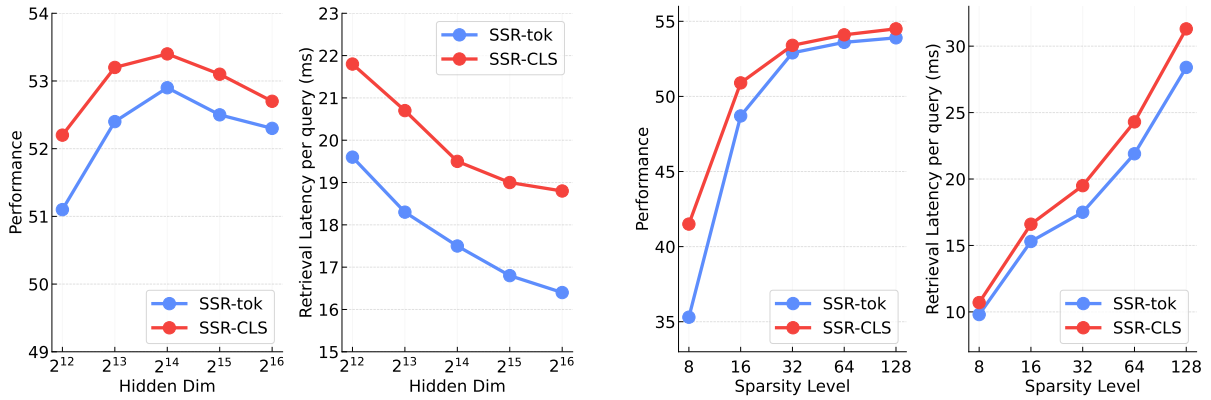


(a) End-to-end analysis. **(b)** Effect of data scale.

Hình C.1: Trái: thời gian huấn luyện, lập chỉ mục và truy hồi trong toàn bộ pipeline. Phải: chất lượng và độ trễ so với ColBERTv2 ở các quy mô dữ liệu khác nhau. Nguồn: Hình 3 của bài báo [1].

C.5. Siêu tham số h, K và trọng số loss

Theo Hình C.2, chất lượng theo h có dạng chữ U ngược với đỉnh ở $h = 2^{14}$: khi h quá lớn so với K, kích hoạt bị phân mảnh khiến các token liên quan ít chia sẻ neuron. Chất lượng theo K có điểm gãy ở $K = 32$: khi $K < 16$ chất lượng giảm mạnh, còn khi K lớn thì độ trễ tăng mà lợi ích không đáng kể. Bảng C.7 cho thấy $K = 32$ ổn định ở hầu hết các miền, còn nhóm tác vụ suy luận nhiều bước hưởng lợi từ $K = 64$; Bảng C.8 cho thấy K thích ứng theo độ dài truy vấn (≤ 3 token dùng $K = 16$; 4–7 dùng $K = 32$; ≥ 8 dùng $K = 64$) đạt chất lượng tương đương $K = 64$ cố định với độ trễ 16,3 ms, khả thi nhờ số hạng Multi-TopK (mục 2.6). Bảng C.9 là khảo sát độ nhạy theo ba trọng số loss được dẫn ở mục 2.7.



(a) Hidden dimension. **(b)** Sparsity level.

Hình C.2: Trái: ảnh hưởng của hidden dimension h (từ 2^{12} đến 2^{16}). Phải: ảnh hưởng của mức sparsity K (từ 8 đến 128). Nguồn: Hình 4 của bài báo [1].

Bảng C.7: nDCG@10 theo mức sparsity K và theo nhóm miền dữ liệu (theo bài báo).

Miền	$K = 8$	$K = 16$	$K = 32$	$K = 64$	$K = 128$
Dữ kiện (DB, FE, SF)	50,4	65,9	67,7	68,2	68,4
Nhiều bước suy luận (NQ, HQ)	50,2	58,6	65,1	66,5	66,8
Khoa học (CV, NF, SD)	29,3	39,6	44,2	44,8	45,3
Ý kiến (AR, TO, FQ)	20,9	33,3	39,5	40,3	40,6

Bảng C.8: K cố định và K thích ứng theo độ dài truy vấn (theo bài báo).

Thiết lập	Cố định 16	Cố định 32	Cố định 64	Thích ứng
nDCG@10 trung bình	50,5	52,9	53,1	53,0
Độ trễ (ms)	16,4	17,5	19,9	16,3

Bảng C.9: Độ nhạy của nDCG@10 trung bình BEIR theo ba trọng số α, β, γ (theo bài báo).

α	1/64	1/32	1/16	0,1	0,2
SSR-tok	52,1	**52,9**	52,4	51,6	50,8
SSR-CLS	52,3	**53,4**	52,7	51,8	51,3
β	0,05	0,1	0,15	0,2	0,25
SSR-tok	52,5	**52,9**	52,8	52,3	51,5
SSR-CLS	53,1	**53,4**	53,1	52,7	51,6
γ	0,01	0,02	0,05	0,1	0,2
SSR-tok	49,7	51,3	**52,9**	50,4	47,6
SSR-CLS	50,3	51,7	**53,4**	51,2	48,5

PHỤ LỤC D. NGUỒN CỦA CÁC SỐ LIỆU HIỆU NĂNG

Bảng D.1 ghi nguồn của từng số liệu hiệu năng dùng trong Chương 4 và 5: file kết quả trong repo nhóm, tập query, số luồng hoặc thiết bị, và phạm vi đo. Đường dẫn tính từ thư mục gốc của repo nhóm; `runs/` là `opt/results/runs/`.

Bảng D.1: Nguồn của các số liệu hiệu năng (nhóm đo).

Số liệu	Ý nghĩa	File; tập query; luồng/thiết bị	Phạm vi đo
1.634,69 ms	Repo exact (chunk 5.000), phần chấm điểm	`opt/logs/fiqa_test100_repo.log`; FiQA test, 100 query đầu; 1 luồng	Query đã mã hóa; không gồm dựng chunk (2.019,2 ms nếu gồm); đỉnh RAM 5,24 GB
2.331,63 ms	Base exact (cấu trúc repo, một chỉ mục)	`opt/logs/fiqa_test100_base.log`; 100 query đầu; 1 luồng	p50 2.103, p95 3.943; đỉnh RAM 24,45 GB
325,7 / 84,1 / 55,2 ms	`fast_hyb` 1 / 6 / 12 luồng	`opt/results/summary.md` (mục "same first 100"); 100 query đầu	Cùng 100 query với hai mốc trên; cơ sở của mọi tỉ số tăng tốc trên FiQA
320,36 / 82,33 / 54,40 ms	`fast_hyb` 1 / 6 / 12 luồng	`opt/logs/fiqa_test_fast.log`; 648 query test	Trung bình; p50 318,5 / 81,6 / 53,0; p95 496,6 / 127,2 / 82,1
58,99 ms	`fast_hyb` 12 luồng trong harness so với PLAID	`results/plaid_bench/ssr.json`; 648 query	Harness khác (`plaid_bench.py`), cùng kernel; chênh với 54,40 ms do overhead gọi hàm
55,88 ms	Exact GPU	`runs/fiqa_test__gpu_exact`; 648 query; RTX 5060	Tới khi top-k về CPU; p95 59,55; 55,9 ms trên 100 query đầu
44,70 ms	ColBERTv2 + PLAID	`results/plaid_bench/plaid.json`; 648 query; GPU	`search` $k = 100$ trên query đã mã hóa; p95 51,1; dev 44,18 ms
36,75 ms	LB-rerank $t = 8$, 1.000 ứng viên	`results/plaid_bench/ssr.json`; 648 query; 12 luồng	p95 57,4; nDCG@10 41,41
173,6 s + 107 s; 24,45 GB	Base exact: dựng posting token toàn corpus theo cấu trúc repo, rồi posting μ; đỉnh RAM	`opt/logs/fiqa_test100_base.log`; `opt/results/summary.json`	Không gồm mã hóa corpus
164,3 s; 2,58 GB	Đường dựng chỉ mục của repo đã sửa; kích thước chỉ mục (258 triệu posting)	`results/plaid_bench/ssr.json`; CPU	Không gồm mã hóa
38,4 s	Dựng lại các chunk ở mỗi lần chạy repo exact	`opt/logs/fiqa_test100_repo.log`	Tính vào 2.019,2 ms/query ở trên
2,92 / 2,6 / 4,79 s; 4,25 GB; 7,45 GB	Dựng chỉ mục `fast_hyb` (forward + μ + posting token) trong ba lần chạy; kích thước chỉ mục; đỉnh RAM	`results/plaid_bench/ssr.json`; `opt/logs/fiqa_dev_sweep.log`; `opt/logs/fiqa_test_fast.log`; 12 luồng	Forward 1,57 GB + μ 0,47 GB + token 2,22 GB
32,6 s; 0,63 GB; 284 s	PLAID: dựng chỉ mục (K-means + nén 2 bit); kích thước; mã hóa corpus	`results/plaid_bench/plaid.json`; GPU	Không gồm mã hóa trong 32,6 s
1.294 s	SSR mã hóa corpus FiQA	`results/plaid_bench/ssr.json`; GPU, batch 16	Ngoại suy từ 5.000 tài liệu đầu
43,97 / 43,99	NFCorpus test: repo exact / base exact và `fast_hyb`	`opt/results/summary.md`; 323 query	Top-100 trùng 100% khi xét hòa điểm; khác do thứ tự các tài liệu bằng điểm
23,18 / 35,86	MS MARCO dev-small MRR@10: SSR-tok / ColBERTv2	`logs/msmarco_rerank.log`; 6.980 query	Re-ranking BM25 top-100
0,22–0,32; 97–99%	Ngưỡng top-100 mỗi token; tỉ lệ tài liệu có cận trên vượt ngưỡng	`opt/results/bound_tightness.json`; 40 query mỗi tập	NFCorpus test, SciFact train

PHỤ LỤC E. HƯỚNG DẪN CHẠY LẠI THÍ NGHIỆM

Các lệnh dưới đây chạy trong thư mục gốc của repo nhóm (https://github.com/duykhanhxx03/ssr-repro-and-optimization) sau khi đã cài môi trường (Bảng 4.6); đường dẫn và tham số đầy đủ ghi trong `README.md` của repo.

E.1. Cấu trúc mã nguồn công bố

Listing E.1: Các thành phần chính của repository SSR và phần tương ứng trong báo cáo.

```
SSR/
  prepare_msmarco.py         # tai va dong goi MS MARCO thanh triplet
  prepare_mteb_eval.py       # tai cac tap BEIR theo dinh dang MTEB
  ssr/
    model.py                 # lop SSR = ColBERT (PyLate) + SparseAutoencoder
    sparse_autoencoder.py    # SAE TopK, AuxK, annealing     -> Chuong 2, muc SAE
    losses.py                # recon, aux, contrastive, CE   -> Chuong 2, muc ham muc tieu
    dataset.py, train.py     # nap triplet MS MARCO, huan luyen
    retrieval/
      inverted_index.py      # dung chi muc nguoc, cham diem  -> Chuong 3, muc chi muc / SSR
      exact_retriever.py     # duong "exact"
      retriever.py           # SparseMaxSimRetriever (SSR++)  -> Chuong 3, muc SSR++
      eval_mteb.py           # CLI: --variant ssr|ssr-cls, --mode exact|pruned
      metrics.py, mteb_io.py # nDCG, MRR; doc qrels           -> Chuong 4, muc do do
  tests/
```

E.2. Cài đặt và chạy

Listing E.2: Cài đặt môi trường, dữ liệu và checkpoint.

```
python -m venv venv && source venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cu128
pip install -e "SSR[eval,data]" pylate sentence-transformers transformers numba scipy pytest
# BEIR (dinh dang MTEB) vao data/mteb/<dataset>/ ; them split train/dev tu BEIR goc
python SSR/prepare_mteb_eval.py --datasets nfcorpus scifact fiqa quora
# Checkpoint SSR-tok
huggingface-cli download Y-Research-Group/SSR --include "SSR-tok/*" --local-dir ckpt
```

Listing E.3: Các thí nghiệm chính (tên mục tương ứng trong Chương 4 và 5).

```
# Ch.4 "Split test": danh gia checkpoint cong bo bang ma da sua (MaxSim theo cong thuc (4))
python -m ssr.retrieval.eval_mteb --task retrieval --variant ssr \
    --model-path ckpt/SSR-tok --dataset nfcorpus --data-dir data/mteb \
    --mode exact --score-device index --split test        # nDCG@10 = 0.4397

# Ch.4 "Doi chung train/dev": theo split va cac mo hinh doi chung
python run_eval.py --ckpt ckpt/SSR-tok --token-scope non-cls -- \
    --task retrieval --variant ssr --dataset scifact --data-dir data/mteb --split train
python eval_dense_colbert.py lightonai/colbertv2.0 scifact train,test
python eval_backbone_dense.py scifact train,test
python quora_ssr.py --dataset quora --splits test,dev --ignore-identical-ids

# Ch.4 "Kiem tra pipeline": ba cach tinh doc lap xac nhan loi cham diem
python crosscheck.py --ckpt ckpt/SSR-tok --data-dir data/mteb --dataset nfcorpus \
    --corpus-npz cache/A/nfcorpus/corpus_32k_test.npz
```

```
python verify_scipy.py nfcorpus cache/A/nfcorpus/corpus_32k_test.npz
python verify_bug.py   nfcorpus cache/A/nfcorpus/corpus_32k_test.npz

# Ch.4 "MS MARCO": BM25 top-100 roi re-rank
python msmarco_rerank.py bm25
python msmarco_rerank.py rerank --models ssr colbertv2 backbone

# Ch.4 "So sanh PLAID" va Ch.5: hieu nang tren FiQA
python plaid_bench.py
python verify_fast_hyb.py nfcorpus test cache/A/nfcorpus/corpus_32k_test.npz
bash opt/run_all.sh fiqa_fast fiqa_sweep
python opt/bench.py --dataset fiqa --split dev --methods fast_hyb fast_ssrpp --threads 6

# Unit test
python -m pytest SSR/tests -q
python opt/tests/test_fast_exact.py
```